RMCT Assignment Guidance 2

Project Proposal - Literature Review & Research Methodology

CT098-3-2 Research Methodology in Computing and Technology (RMCT)
Assignment Part 2 - 45% of Overall Module Assessment

2026

Contents

Contents 2
About This Guide 5
Assignment Recap 5
Mapping the Assignment Guidance to Project Proposal Part 2 7
Thinking Like a Computing Researcher 10
Learning Outcomes 10
1.1 What is Research in Computing and Technology? 10
1.2 Why is Research Important in Software Development? 12
1.3 RMCT as the Foundation of the Final Year Project 12
1.4 Research Throughout the Software Development Life Cycle (SDLC) 13
1.5 Running Example: Smart Attendance System for Techspire College 16
1.6 Thinking Like a Computing Researcher 17
1.7 The RMCT Philosophy 17
Chapter Summary 18
Transition to Chapter 2 18
Exploring the Research Domain 19
Learning Outcomes 19
2.1 What is a Literature Review? 21
2.2 Reverse the Journey: From Project Title to Research Domains 21
2.3 Expanding Project Concepts into Research Domains 23
2.4 The Literature Review Funnel 27
2.5 Why is the Literature Review Important? 28
2.6 Understanding Research Domains 28
2.7 Searching for Academic Literature 29
2.8 Using Boolean Operators 30
2.9 Finding Credible Sources 30
2.10 Evaluating the Quality of Literature 31
2.11 Organising the Literature 32
2.12 Running Example 33
Chapter Summary 33
Transition to Chapter 3 33
Learning from Existing Knowledge and Systems 34
Learning Outcomes 34
3.1 From Literature Review to Literature Synthesis 34
3.2 Summarising vs Synthesising Literature 35
3.3 Learning from Existing Software Systems 35
3.4 Identifying Similar Systems 36
3.5 Comparing Similar Systems 36
3.6 Choosing proper evaluation criteria : Analyzing Strength and Weaknesses 36
3.7 Building Evidence for Your Proposed System 38
3.8 Running Example 39
3.9 Preparing for Research Gap Identification 39
Common Mistakes 39
Best Practice 40

Chapter Summary 40
Transition to Chapter 4 40
Finding the Research Gap 41
Learning Outcomes 41
4.1 What is a Research Gap? 41
4.2 Where Do Research Gaps Come From? 41
4.3 Types of Research Gaps in Computing and Technology 42
4.4 Identifying Research Gaps from Literature 43
4.5 Identifying Research Gaps from Similar Systems 43
4.6 Running Example 43
4.7 Justifying the Proposed Solution 44
4.8 Demonstrating Research Contribution 44
Common Mistakes 45
Best Practice 45
Chapter Summary 45
Transition to Chapter 5 45
Making Evidence-Based Technical Decisions 46
Learning Outcomes 46
5.1 Why Technical Decisions Matter 46
5.2 Research as the Foundation for Technical Decisions 47
5.3 Common Technical Decisions in Software Projects 47
5.4 Evaluating Technology Alternatives 48
5.5 Running Example 48
5.6 Aligning Technical Decisions with User Requirements 49
5.7 Justifying Technical Decisions 49
5.8 From Research to Software Engineering Decisions 50
5.9 Preparing for Research Methodology 50
Common Mistakes 50
Best Practice 50
Chapter Summary 51
Transition to Chapter 6 51
Planning the Research Methodology 54
Learning Outcomes 54
6.1 Why Research Methodology Matters 55
6.2 Understanding the Methodology Hierarchy 56
6.3 Proposed System Development Methodology 58
6.4 Research Design 66
6.5 Research Approach 69
6.6 Target Users 70
6.7 Sampling Strategy 72
6.8 Data Collection Methods 74
6.9 Ethical Considerations 75
6.10 Running Example : Bringing the Methodology Together 77
Common Mistakes 78
Best Practice 78
Chapter Summary 79
Transition to Chapter 7 79
Collecting Evidence from Users 80
Learning Outcomes 80
7.1 Why Evidence Collection Matters 80

7.2 Selecting Appropriate Data Collection Methods 83
7.3 Designing Effective Questionnaires 84
7.4 Conducting Effective Interviews 87
7.5 Using Observation to Understand Current Processes 88
7.6 Document Analysis 88
7.7 Combining Multiple Methods (Triangulation) 89
7.8 Designing Research Instruments 89
7.9 Ensuring Data Quality 89
7.10 Recognising Research Bias 90
7.11 Running Example 93
Common Mistakes 94
Chapter Summary 94
Transition to Chapter 8 94
Connecting Research to Software Development 95
Learning Outcomes 95
8.1 The Complete Research Journey 95
8.2 How Research Supports the Software Development Life Cycle 97
8.3 Transforming Research into Software Requirements 97
8.4 Running Example: The Complete Journey 98
8.5 RMCT as the Foundation of the Final Year Project 99
8.6 The Philosophy of Research in Computing and Technology 100
Common Mistakes 100
Best Practice 100
Final Chapter Summary 101
Final Reflection 101

About This Guide

This guide accompanies RMCT Assignment Guidance 1 (Project Proposal - Introduction) and provides complete instructional coverage of Assignment Part 2: the Literature Review and Research Methodology sections of your Project Proposal.

The guide is organised into eight teaching chapters, each demonstrated throughout using a single running example: the Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College.

RMCT PHILOSOPHY

Research in Computing and Technology is a continuous, evidence-based process of investigating problems, understanding users, analysing existing knowledge, making informed technical decisions, developing software solutions, and evaluating whether those solutions effectively solve real-world problems.

CORE EDUCATIONAL PRINCIPLE

Assignment Guidance 1 teaches students how to move from a real-world problem to a proposed software solution. Assignment Guidance 2 teaches students to reverse that journey by expanding the project title into broader research domains, reviewing existing knowledge from general to specific, comparing alternative technologies, identifying research gaps, and justifying every aspect of the proposed solution using academic evidence. In RMCT, the project title is not the starting point for a literature search; it is the destination of a structured, evidence-based research journey.

Assignment Recap

Component	Weight	Focus
Part 1 - Project Proposal (Introduction)	40%	Title, Abstract, Problem Statement, Aim, Objectives, Research Questions, Significance, Proposed System Overview
Part 2 - Project Proposal (this guide)	45%	Literature Review (Research Domains, Similar Systems) and Research Methodology (Software Development Methodology, Research Design, Research Approach, Target Users, Sampling, Data Collection, Ethical Considerations)
Presentation	15%	A short presentation defending the completed Research Concept Paper

Formatting requirements: maximum 3,500 words (excluding references), the paper template provided in the Assignment Description, 1.0 line spacing, Times New Roman size 12, and APA 7th Edition referencing.

Mapping the Assignment Guidance to Project Proposal Part 2

The eight chapters in this guide are designed as learning chapters, not report headings. Each chapter teaches the knowledge and skills required to complete specific sections of Project Proposal Part 2 (Literature Review and Research Methodology). Students should draw on the guidance from multiple chapters when writing each section of their proposal, rather than treating any single chapter as a direct template for a single report section.

General Structure of Assignment Part 2 (Official Report Structure)

Before working through the eight teaching chapters, students should first be familiar with the actual section headings required in the Project Proposal Part 2 report, as specified in the Assignment Description. The table below summarises the official recommended structure of the Research Concept Paper for this assignment.

Report Section	Required Content
1. Literature Review	1.1 Research Domains; 1.2 Similar Systems
2. Research Methodology	2.1 Identify Target User; 2.2 Sampling Method; 2.3 Data Collection Method
3. Conclusion	Summary of the proposal and its overall significance
4. References	All sources cited, in APA 7th Edition format

The paper template provided in the Assignment Description carries this structure over into the actual report, in which the Introduction from Part 1 is retained and the Literature Review and Methodology sections are numbered as follows:

Paper Template Section	Subsections	Suggested Word Count (approximately)
1. Introduction	No subheadings (carried over from Part 1)	Does not count
2. Literature Review	2.1 Domain 1; 2.2 Domain 2; 2.3 Domain 3; … 2.n Domain n ; 2.(n+1) Similar Systems	2250
3. Methodology	3.1 Proposed Software Development Methodology; 3.2 Research Design; 3.3 Research Approach; 3.4 Target Users; 3.5 Sampling Strategy; 3.6 Data Collection Methods; 3.7 Ethical Considerations	1100
4. Conclusion	No subheadings	150
5. References	No subheadings	Does not count
	Total	3500

KEY MESSAGE

This is the required report skeleton for Assignment Part 2. The eight chapters of this guide teach the knowledge and thinking needed to write these sections well; they are learning chapters, not additional report sections. Check with your lecturer for optional sections, variations, or additional sections that may be required.

Chapter-by-Chapter Mapping

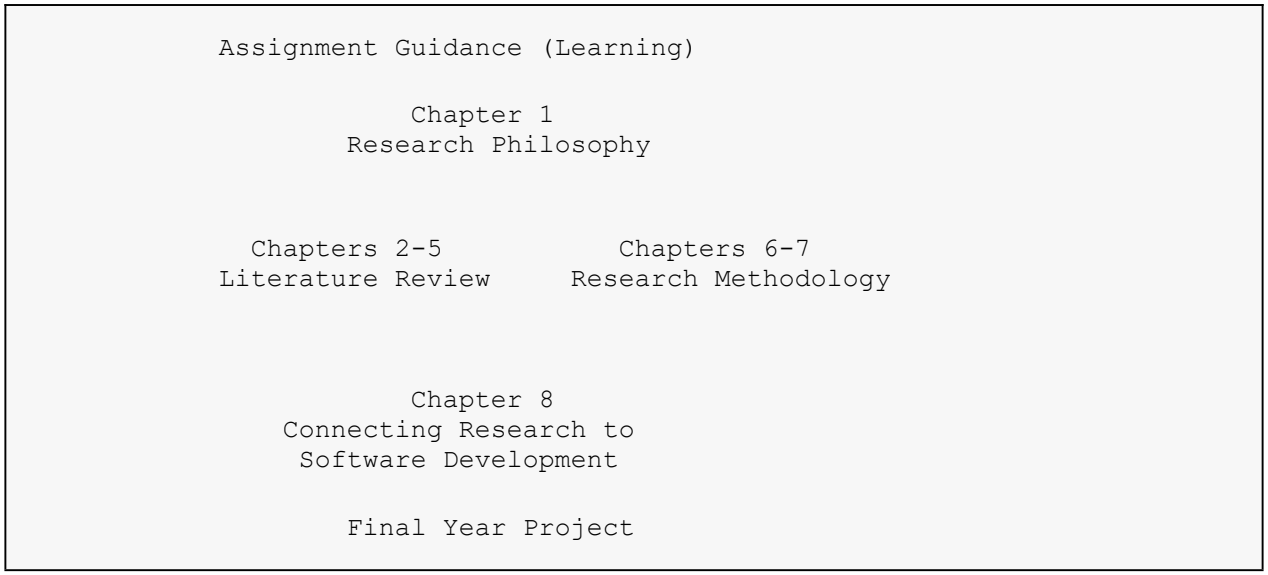
Assignment Guidance Chapter	**Purpose**	**Supports Proposal Section(s)**
Chapter 1 - Thinking Like a Computing Researcher	Introduces the philosophy of research in Computing and Technology and explains how research supports software engineering throughout the Software Development Life Cycle (SDLC).	Provides the overall research mindset that underpins the entire proposal, particularly the Literature Review and Research Methodology chapters.
Chapter 2 - Exploring the Research Domain	Teaches students how to search academic databases, identify relevant literature, evaluate sources, and explore current technologies and industry trends.	Literature Review
Chapter 3 - Learning from Existing Knowledge and Systems	Explains how to critically analyse research papers, existing software systems, technologies, and related work to identify strengths, limitations, and best practices.	Literature Review
Chapter 4 - Finding the Research Gap	Demonstrates how to identify research gaps, justify the proposed project, and establish the need for the proposed software solution.	Literature Review (Research Gap, Problem Justification and Proposed Solution)
Chapter 5 - Making Evidence-Based Technical Decisions	Guides students in selecting and justifying appropriate technologies, software architecture, development tools, databases, frameworks, and security mechanisms using evidence from the literature.	Literature Review (Technology Review and Technical Justification)
Chapter 6 - Planning the Research Methodology	Explains how to develop the Research Methodology, including the Software Development Methodology, Research Design, Research Approach, Target Users, Sampling Strategy, Data Collection Methods, and Ethical Considerations.	Research Methodology
Chapter 7 - Collecting Evidence from Users	Provides detailed guidance on questionnaires, interviews, observations, document analysis, research instruments, pilot testing, data quality, and reducing research bias.	Research Methodology (Data Collection Methods, Research Instruments, Data Collection Process)
Chapter 8 - Connecting Research to Software Development	Integrates all previous chapters and demonstrates how literature review and research methodology lead to requirements engineering, system design, software development, testing, evaluation, and the Final Year Project (FYP).	Supports both Literature Review and Research Methodology, and explains how the proposal becomes the foundation for the Final Year Project (FYP).

How the Chapters Map to the Proposal

The relationship between the teaching chapters and the actual proposal structure can be summarised as follows:

Proposal Section	Primary Supporting Chapters
Literature Review	Chapters 2, 3, 4, and 5, with the research philosophy established in Chapter 1
Research Methodology	Chapters 6 and 7, supported by the research philosophy established in Chapter 1
Transition to Final Year Project (FYP)	Chapter 8

Visual Learning Path



This distinction reinforces the educational purpose of the handbook: the guidance chapters teach the underlying concepts and ways of thinking, while the Project Proposal applies those concepts within the report structure required by the Assignment Description.

KEY MESSAGE

Students should not think, "I have to write Chapter 2 of my report." Instead, they should think, "I should study Chapters 2-5 of this guidance to write the Literature Review, and Chapters 6-7 to write the Research Methodology."

Rather than approaching each proposal section as an isolated writing task, students should treat it as the application of reasoning developed across several chapters of this guide, grounded throughout in the research philosophy introduced in Chapter 1 and brought full circle in Chapter 8, where the proposal is connected to the Final Year Project.

CHAPTER 1

Thinking Like a Computing Researcher

Chapter Purpose

This chapter introduces the fundamental philosophy of Research Methods in Computing and Technology (RMCT). It explains why research is an essential component of software engineering and demonstrates how research supports every stage of software development. Students will also understand how RMCT serves as the foundation for the Final Year Project (FYP), preparing them to investigate real-world problems and develop evidence-based software solutions.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the purpose of research in Computing and Technology.
- Describe how research supports every phase of software development.
- Differentiate computing research from traditional academic research.
- Explain the relationship between RMCT and the Final Year Project (FYP).
- Recognise the importance of evidence-based decision making in software engineering.

1.1 What is Research in Computing and Technology?

Research in Computing and Technology is not simply the process of writing reports or reviewing academic literature. Instead, it is a systematic and evidence-based approach to understanding problems, analysing existing knowledge, identifying user needs, making informed technical decisions, developing software solutions, and evaluating whether those solutions effectively address real-world problems.

Unlike research in many other disciplines, computing research is solution-oriented. Its primary objective is not only to generate knowledge but also to apply that knowledge to design, develop, implement, and evaluate technology-based solutions that solve practical problems. This aligns directly with the RMCT module, where students investigate a problem and propose a software artefact as the intended solution.

KEY MESSAGE

In Computing and Technology, research is not performed for its own sake. It is conducted to support the development of better software systems.

Redefining Research for Software Developers

Traditional definitions describe research as "a systematic investigation" or "collecting and analysing data." These definitions are not wrong, but they do not explain what research

means for a software engineer building a real system. A more useful definition for this guide is:

SOFTWARE DEVELOPMENT DEFINITION
Research is the systematic process of reducing uncertainty by gathering, evaluating, and applying evidence to make better software engineering decisions throughout the software development lifecycle.

Two short statements capture the same idea and are worth remembering throughout this guide:

KEY MESSAGE
Research means replacing assumptions with evidence.

KEY MESSAGE
Research is not just about finding answers. It is about making better decisions with evidence.

Held together, these statements point to the single idea that runs through every chapter of this guide: research is not a chapter of the proposal. It is the continuous process that guides every important software engineering decision using evidence.

1.2 Why is Research Important in Software Development?

Many students mistakenly believe that software development begins with programming. In reality, successful software projects begin with understanding the problem before developing the solution.

Research helps software developers to:

- understand real-world problems;
- identify the needs of users and stakeholders;
- analyse existing systems and technologies;
- identify research gaps and opportunities for improvement;
- justify software design and technology choices using evidence;
- evaluate whether the developed system successfully addresses the identified problem.

Without proper research, software solutions are often based on assumptions rather than evidence, increasing the likelihood of developing systems that fail to meet user needs.

BEST PRACTICE
Every major software engineering decision should be supported by evidence rather than personal assumptions or preferences.

Generic Research vs Computing Research

Although all research follows systematic and ethical principles, its purpose varies across academic disciplines. The comparison below distinguishes generic academic research from research in Computing and Technology.

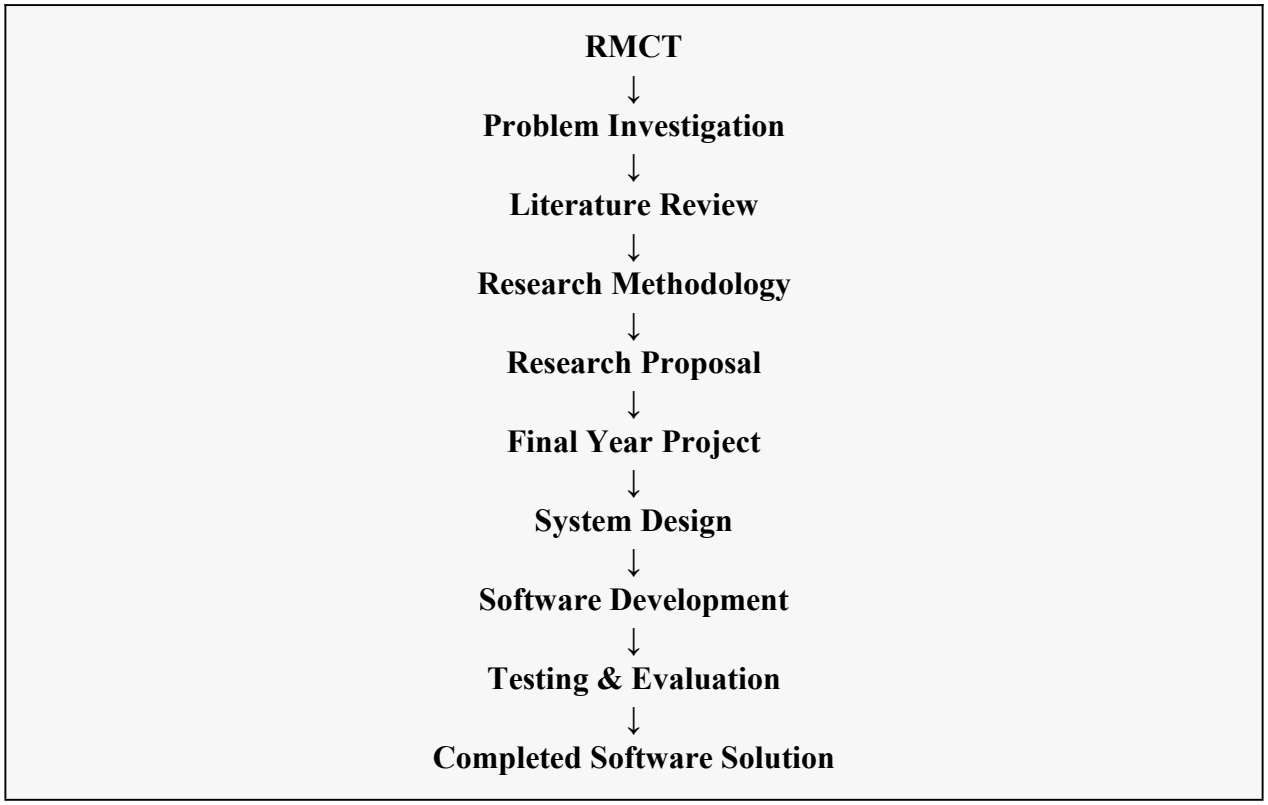
Generic Basic Research	Computing Research
Seeks to answer research questions	Seeks to solve real-world problems through technology
Produces new knowledge	Produces justified software solutions
Emphasises theory development	Emphasises software development supported by evidence
Focuses on investigation	Focuses on investigation and implementation
Ends with research findings	Continues through development, testing, and evaluation

KEY MESSAGE
Computing research does not stop after understanding the problem. It continues until the proposed solution has been designed, developed, tested, and evaluated.

1.3 RMCT as the Foundation of the Final Year Project

Research Methods in Computing and Technology is more than an academic module - it is the foundation of the Final Year Project (FYP). The proposal developed during RMCT becomes

the blueprint for the software system students will later design, implement, test, and evaluate during their FYP.



Students should therefore treat their proposal not merely as an assignment but as the first stage of their software development journey.

1.4 Research Throughout the Software Development Life Cycle (SDLC)

Research is often misunderstood as an activity completed before software development begins. In practice, research supports every phase of the Software Development Life Cycle (SDLC).

SDLC Phase	How Research Contributes
Planning	Understand the problem and define project scope.
Requirements Analysis	Gather evidence from users and stakeholders to identify functional and non-functional requirements.
System Design	Evaluate architectures, technologies, databases, security mechanisms, and design alternatives.
Development	Apply research findings and justified technical decisions during implementation.
Testing	Collect user feedback and evaluate whether the system satisfies requirements.
Deployment & Evaluation	Assess system effectiveness, identify limitations, and recommend future improvements.

Continuous Research
↓
Planning → Analysis → Design → Development → Testing → Deployment → Evaluation

KEY MESSAGE
Research is a continuous process that informs, justifies, supports, and evaluates every stage of software engineering.

Uncertainty is the Heart of Research

Every phase of the software development lifecycle begins with a form of uncertainty. Research is the activity that reduces that uncertainty before the next phase can proceed with confidence:

Unknown Problem
↓
Problem Research
↓
Unknown Solution
↓
Literature Review
↓
Unknown Requirements
↓
Requirements Gathering
↓
Unknown Design
↓
Design Research
↓
Unknown Implementation
↓
Development Research
↓
Unknown Quality
↓
Testing & Evaluation
↓
Improved Understanding

Notice that research never disappears from this sequence. Only the questions change: early on, research asks "what is the problem?"; later, it asks "does this design work?"; later still, it asks

"does this system perform as expected?" A student who understands this will recognise research activity in every chapter of their project, not only in the Literature Review chapter.

Research is a Cycle, Not a Single Step

Students often imagine the research process as a straight line that ends once the Literature Review chapter is written. In practice, research behaves more like a cycle that keeps refining understanding as new evidence appears:

Literature
↓
Requirements
↓
New Evidence
↓
Literature (revisited)
↓
Better Understanding
↓
Design
↓
Testing
↓
Unexpected Findings
↓
Literature Again
↓
Improved Solution

Understanding continuously improves as the project moves forward. A finding from a requirements interview can send a student back to the literature; an unexpected result during testing can do the same. This is normal, expected, and is in fact a sign that research is being taken seriously rather than treated as a formality.

"Re-search": A Useful Way to Remember This

As a memorable way of thinking about this idea, rather than as a strict definition of the word, it helps to notice that the word "research" contains "search" twice. Whenever new evidence contradicts an earlier assumption, a good researcher simply searches again:

Initial literature: Facial Recognition
↓
Requirements Gathering
↓
Finding: Privacy concerns raised
↓

Search again
↓
Privacy-preserving attendance approach
↓
Improved solution

KEY MESSAGE
Research is never embarrassed to change direction. Searching again when the evidence demands it is good research practice, not a mistake to be hidden.

1.5 Running Example: Smart Attendance System for Techspire College

Throughout this guidance, a single running example will be used to demonstrate how research supports software development.

Current Situation

Techspire College already uses an eDigital Attendance System, allowing attendance records to be stored digitally. However, lecturers must still manually mark attendance for every student during each class. This results in several challenges:

- Time-consuming attendance marking.
- Interruption to teaching and learning.
- Possibility of proxy attendance.
- Limited automation.
- Limited real-time analytics and reporting.
- Increased administrative workload for lecturers.

Proposed Solution

To address these limitations, the proposed project is a Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College. The proposed system aims to:

- automatically recognise students using facial recognition;
- record attendance without manual interaction;
- reduce opportunities for proxy attendance;
- provide real-time attendance analytics and reporting;
- reduce lecturer workload; and
- integrate with the existing digital attendance process.

This running example will be revisited throughout the remaining chapters to demonstrate how research supports each stage of software development.

1.6 Thinking Like a Computing Researcher

A computing researcher approaches problems differently from someone who immediately begins designing or programming a solution. Instead of asking "How can I build this system?", a computing researcher first asks:

- What is the actual problem?
- Who experiences this problem?
- Why does the problem exist?
- What evidence supports its existence?
- How have others attempted to solve it?
- What limitations remain?
- How can a better software solution be developed?

These questions encourage critical thinking and ensure that software solutions are developed based on evidence rather than assumptions.

COMMON MISTAKE

Many students begin designing software before fully understanding the problem. Effective software engineering begins with investigation, analysis, and evidence-based decision making.

1.7 The RMCT Philosophy

RMCT PHILOSOPHY

Research in Computing and Technology is a continuous, evidence-based process of investigating problems, understanding users, analysing existing knowledge, making informed technical decisions, developing software solutions, and evaluating whether those solutions effectively solve real-world problems.

This philosophy should guide every section of the proposal and every stage of the software development process.

From Project Title to Research Domains

Assignment Guidance 1 concluded with a project title, for example, Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College. That title is the destination of the first stage of the research journey, not the starting point of the next one. Chapter 2 will show how to work backwards from this title, breaking it into its major concepts and expanding those concepts into the broader research domains that must be explored before the literature review can begin.

Every project title follows a consistent structure: Solution + Need + Context. Recognising this structure matters because each component represents a decision reached through the

reasoning process in Assignment Guidance 1, and each one must be recovered and justified using academic evidence in this chapter.

KEY MESSAGE
The project title is not where the literature review begins. It is the destination where the literature review is heading.

Chapter Summary

At the end of this chapter, students should understand that RMCT prepares students for the Final Year Project; research is an essential component of software engineering; research supports every stage of the Software Development Life Cycle (SDLC); software engineering decisions should be justified using evidence; and a research proposal is not merely an academic report, it is the foundation for developing a successful software solution.

Transition to Chapter 2

Having understood the role of research in software engineering, the next step is to explore existing knowledge within the chosen research area. Chapter 2: Exploring the Research Domain will explain how to identify research domains, search academic literature effectively, evaluate credible sources, and establish a strong foundation for the literature review.

CHAPTER 2

Exploring the Research Domain

Chapter Purpose

A strong software solution begins with a thorough understanding of the existing body of knowledge surrounding the problem. This chapter explains how to identify an appropriate research domain, search for credible academic literature, evaluate the quality of sources, and organise existing knowledge into meaningful themes. Students will learn how a literature review supports evidence-based software development by helping them understand current technologies, existing approaches, and emerging trends before proposing their own solution.

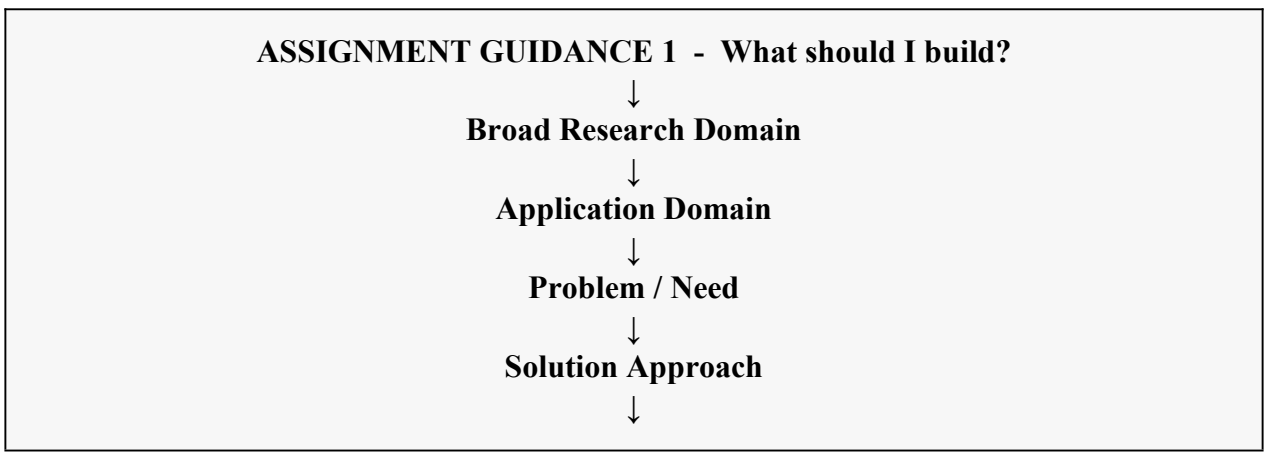
Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the purpose of a literature review in Computing and Technology.
- Identify relevant research domains related to a software project.
- Search academic databases effectively using appropriate keywords.
- Evaluate the credibility and relevance of different information sources.
- Organise literature into logical research themes.
- Explain how literature supports evidence-based software development.

The Complete Research Journey

The RMCT module answers three questions in sequence, and the diagram below shows how they connect into a single, continuous research journey. Assignment 1 answers **what should I build?** by moving forward from a real-world context to a project title. First part of Assignment 2, i.e. Literature Review, then answers **why is this the right solution?**, reversing direction to recover and justify the thinking behind that title using academic evidence. Finally, second part of Assignment 2, i.e. Methodology, answers **how will I build and evaluate it?**, planning the development and research activities that carry the justified solution through to the Final Year Project.



Proposed Technology

↓

Existing Information Systems

↓

Project Title

↓

——————

↓

ASSIGNMENT GUIDANCE 2: LITERATURE REVIEW - Why is this the right solution?

↓

Recover Original Thinking

↓

Context (Broad to Application Domain)

↓

Need

↓

Solution

↓

Academic Evidence

↓

Similar Systems & Technology Comparison

↓

Research Gap

↓

Justified Solution

↓

——————

↓

ASSIGNMENT GUIDANCE 2: RESEARCH METHODOLOGY - How will I build and evaluate it?

↓

Proposed Software Development Methodology

↓

Research Design

↓

Target Users

↓

Sampling Strategy

↓

Data Collection Methods

↓

Ethical Considerations

↓

Software Development

↓

Evaluation
↓
Final Year Project

RESEARCHER'S THINKING
Before I decided this word belonged in my project title, what did I need to understand first?

2.1 What is a Literature Review?

A literature review is a systematic examination of existing knowledge related to a research problem. Rather than simply collecting references, it enables researchers to understand what has already been studied, identify current technologies and approaches, recognise limitations in existing work, and establish a strong foundation for their proposed solution.

In Computing and Technology, the literature review plays an essential role in software development because it helps researchers make informed technical decisions based on existing evidence instead of assumptions.

KEY MESSAGE
A literature review is not about summarising articles, it is about understanding existing knowledge to build a better software solution.

2.2 Reverse the Journey: From Project Title to Research Domains

Assignment Guidance 1 taught a forward journey: starting from a real-world context, identifying a problem, establishing a need, and arriving at a proposed solution expressed as a project title. Many students assume the literature review should follow the same direction, so they begin by searching Google Scholar for their exact project title, or for a phrase such as "Smart Attendance System." This is poor research practice, because it starts from the most specific point in the research landscape rather than the broadest.

Researchers do the opposite. Before narrowing down to a specific solution, they first explore the wider body of knowledge surrounding the problem. In Assignment Guidance 2, students are taught to reverse the journey used in Assignment Guidance 1: instead of moving from context to solution, they move from the solution back through major concepts, research domains, literature, and evidence, before arriving again at a well-justified version of their proposed system.

In practice, this reversal begins by recovering the original thinking behind the title: the context, the need, and the solution that were reasoned out in Assignment Guidance 1. Once that thinking has been recovered, each part can be expanded into major concepts and research domains, and then defended with academic evidence, so that the title is no longer simply asserted but justified.

KEY MESSAGE

The project title is not the starting point of the literature review. It is the destination reached after understanding the broader body of knowledge.

Solution (Project Title)
↓
Major Concepts
↓
Research Domains
↓
Literature Search
↓
Existing Knowledge
↓
Existing Systems & Technology Comparison
↓
Research Gap
↓
Evidence-Based Technical Decisions
↓
Research Methodology
↓
Software Development

Recovering the Original Thinking Process

Before identifying research themes, it helps to remember how the project title was created in the first place. Techspire College was never the starting point of the project. You did not wake up thinking about Techspire College; you started from a much broader context and progressively narrowed it down until a specific project, at a specific institution, became clear.

For the running example, that narrowing process looked like this:

Digital Transformation in Higher Education
↓
Student Attendance Management
↓
Student Lecture Attendance
↓
Techspire College
↓
Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College

Techspire College is the final destination of topic formulation. It is not the beginning of research.

Once the project title has been created, research begins by reversing this same reasoning. Students simply recover the broader concepts that originally led them to the project context:

Techspire College
↓
Student Lecture Attendance
↓
Student Attendance Management
↓
Digital Transformation in Higher Education

Only after recovering these broader concepts should the literature search begin. This is the philosophical bridge between Assignment Guidance 1 and Assignment Guidance 2.

KEY MESSAGE
I already know where my research themes come from. My project title was not created randomly; I narrowed a broad context until I identified my project, and now I simply reverse that thinking to recover it.

2.3 Expanding Project Concepts into Research Domains

The first step in reversing the journey is to break the project title down into its major concepts, and then expand each concept into a broader research domain. This produces the structure that the literature review will be organised around.

Project Title: Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College
↓
Major Concepts/Conceptual units
↓
Techspire College • Attendance Management • Improve Student Lecture Attendance • Facial Recognition • Smart System

Project Concept	Research Domain
Techspire College	Higher Education Institute to Digital Transformation in Education Institutes
Student Attendance Management	Attendance Management
Improve Student Lecture Attendance	Challenges in managing attendance
Facial Recognition	Biometric Authentication / Computer Vision

Project Concept	Research Domain
Smart System	Artificial Intelligence in Attendance Management

Notice that the research domains are broader than the words used in the title. Students search literature using these research domains, not simply the words inside the project title.

BEST PRACTICE
When expanding a project title, write down every major concept first, then ask: what wider field of research does this concept belong to? That wider field is the research domain.

Universal Computing Research Hierarchy

Every RMCT project is fundamentally an Information System. Therefore, every literature review should naturally move through the following hierarchy:

Level	Purpose
Research Domain	Broad field of knowledge
Application Domain	Business or organisational function
Problem / Need	Operational challenge requiring improvement
Business Solution	Information System that supports the business function
Technical Solution	Technologies enabling the Information System
Existing Information Systems	Previous implementations of the proposed solution

This hierarchy is the conceptual foundation of this chapter. Notice that it separates the Business Solution, the Information System itself, from the Technical Solution, the technologies that enable it, echoing the distinction between an Information System and a Technology explained later in this section.

The Universal Literature Review Formula

Because the minimum scope of Final Year Project for your study program is that it should be either a Web/Mobile/Desktop (PC) Application, every project is fundamentally an Information System designed to improve a real organisational or business process. This means that beneath almost every project title lies the same underlying research structure. Rather than inventing research themes from scratch, students can move through six universal layers of computing research:

Layer	Purpose	General Formula	Example
1	Broad Research Domain	Digital Transformation in [Domain]	Digital Transformation in Higher Education
2	Application Domain	Business / Operational Area	Student Attendance Management

Layer	Purpose	General Formula	Example
3	Problem / Need	Problems or Challenges in the Operational Area	Challenges in Managing Student Lecture Attendance
4	Solution Approach	General Approaches Used by Information Systems to Solve the Problem	Biometric Authentication for Attendance Management
5	Proposed Technology	Specific Technology Used in the Proposed System	Facial Recognition Technology
6	Existing Information Systems	Existing Systems Implementing the Technology	Facial Recognition-Based Attendance Management Systems

This becomes the universal thinking framework for identifying literature review themes for almost any RMCT project.

Why This Framework Works

These six layers are not arbitrary categories. They represent the natural way computing researchers progressively narrow their investigation:

Broad Research Domain
↓
Application Domain
↓
Problem / Need
↓
Solution Approach
↓
Proposed Technology
↓
Existing Information Systems

Working through these layers in order corresponds to the Universal RMCT Literature Review Formula below, where each stage is guided by a single question:

Stage	Investigate	Guiding Question
1	Digital Transformation in the Domain	How is this domain becoming digital?
2	Business / Operational Function	Which organisational process is being improved?
3	Existing Problems / Needs	Why does this process require improvement?
4	Approaches that Information Systems take Supporting the Function	How do Information Systems address these problems?
5	Enabling Technologies	Which technologies power these Information Systems?
6	Existing Information Systems	How have previous systems implemented these technologies?

This framework should become the generic thinking model applied to almost every RMCT project.

Critical Computing Research Principle: Information System vs. Technology

Students frequently confuse an Information System with a Technology. Artificial Intelligence, Blockchain, Facial Recognition, Machine Learning, Cybersecurity, IoT, Cloud Computing, QR Codes, and NFC are not, by themselves, the solution.

KEY MESSAGE

The actual solution is the Information System that improves a real organisational/societal process. The chosen technology is simply the enabling technology.

For the running example, the Smart Facial Recognition-Based Attendance Management System is the Information System, and it is the actual proposed solution. Facial Recognition is merely the enabling technology chosen to implement it. Keeping this distinction clear helps students avoid proposals that read as if the technology itself is the project, rather than the organisational/operational problem the technology is being used to solve.

From Recovering Context to Identifying a Research Gap

Bringing this together, the intellectual journey through Chapter 2 should feel like a single continuous thought:

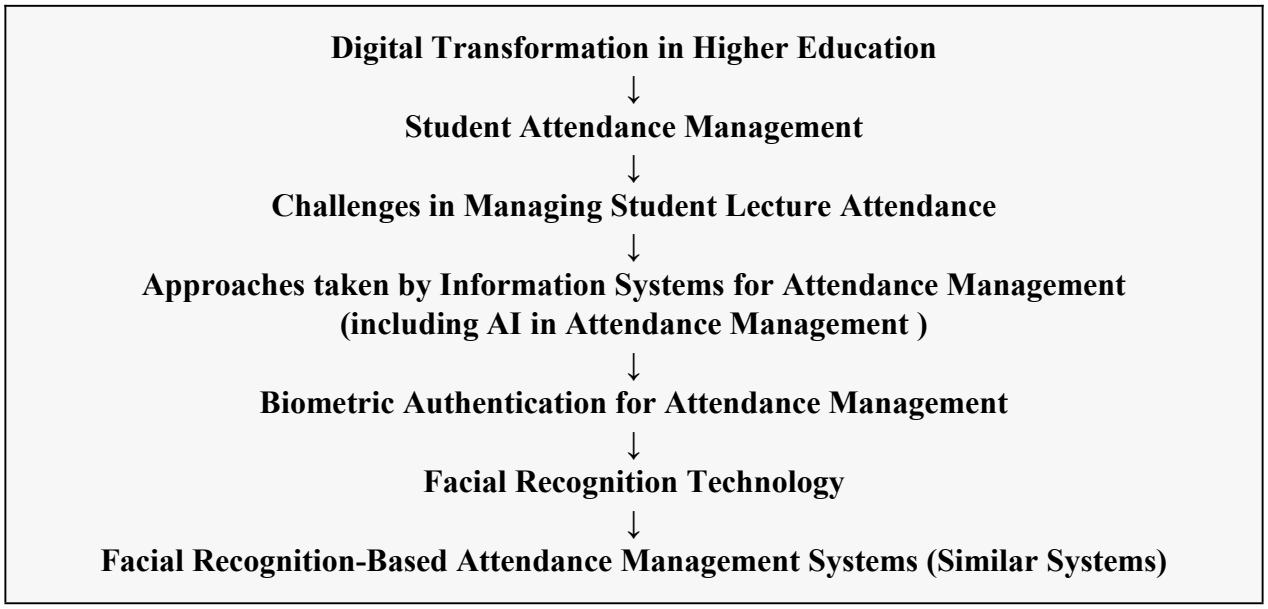
- My project title was not created randomly.
- I narrowed a broad context until I identified my project.
- Now I simply reverse that thinking.
- I recover the broader research domains.
- I understand the organisational/operational problems.
- I study the approaches that Information Systems use to solve it.
- I investigate enabling technologies.
- I analyse existing Information Systems.
- Now I can identify meaningful research gaps.

KEY MESSAGE

Students should no longer guess research themes. Instead, they systematically derive them using the universal framework above, the same way they systematically derived their project title in Assignment Guidance 1.

2.4 The Literature Review Funnel

Once research domains have been identified, the literature review should move progressively from these broad domains toward the specific proposed solution. This is known as the literature review funnel.



Notice the important refinement here. Students do not begin by searching for "Attendance Systems" or "Facial Recognition Systems." They first understand the domain, the organisational process, the operational problem, and the general approaches taken by Information Systems, then the enabling technology, and only then existing implementations. This preserves the educational separation between Chapter 2 (Exploring Research Domains) and Chapter 3 (Learning from Existing Knowledge and Systems).

Reading the funnel from top to bottom shows how a literature review should be structured: it begins with the broadest relevant field and narrows step by step until it reaches the specific technologies and system proposed. Reading it from bottom to top shows why the proposed solution should never be the starting point of the search: it is the most specific point in a much larger body of knowledge.

KEY MESSAGE

A literature review moves from general to specific. It does not start with the proposed solution and work outward; it starts broad and narrows down until the proposed solution emerges as the logical next step.

2.5 Why is the Literature Review Important?

A well-conducted literature review helps students to:

- Understand the current state of research.
- Identify technologies commonly used to solve similar problems.
- Learn from previous research findings and experiences.
- Recognize strengths and limitations of existing solutions.
- Justify design and technology choices using evidence.
- Identify research opportunities and gaps.

KEY MESSAGE
Without reviewing existing knowledge, researchers risk developing software that duplicates previous work or overlooks proven approaches.

2.6 Understanding Research Domains

A research domain is the broader field of knowledge in which a project is situated. As demonstrated in the concept-expansion example above, research domains are not simply the words that appear in the project title; they are the wider academic and professional fields those words belong to.

Research domains have three important characteristics. They are broader than keywords, since a single concept such as facial recognition belongs to a wider field such as Biometric Authentication or Computer Vision. They are not random topics, since every domain must be clearly connected to a concept extracted from the project title. And they are not copied directly from the title, since copying the title only reproduces the most specific point in the research landscape rather than the field surrounding it.

Most software projects span multiple domains because real-world problems often involve different technologies, techniques, and application areas. For the running example, Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College, the concept-expansion table combines into the following set of research domains:

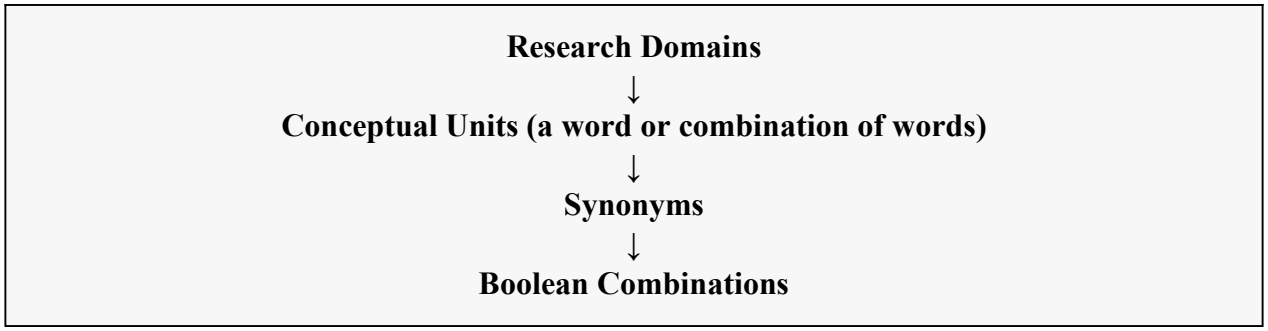
- Facial Recognition / Computer Vision
- Artificial Intelligence
- Biometric Authentication
- Attendance Management
- Higher Education Information Systems
- Human-Computer Interaction

- Data Privacy and Security

Each domain contributes different knowledge that helps shape the proposed software solution, and together they provide the structure around which the entire literature review is organised.

2.7 Searching for Academic Literature

Search keywords should not be copied directly from the project title. Instead, effective literature searching follows the same reverse logic used to identify research domains: keywords are derived by moving from research domains, to the concepts within them, to synonyms for those concepts, and finally to Boolean combinations of those synonyms.



For example, the domain Biometric Authentication produces the concept facial recognition, which can be expressed using synonyms such as face recognition or facial identification, and combined with related concepts such as attendance management using Boolean operators.

Example keywords derived this way for the running project: Facial Recognition, Face Recognition, Attendance Management System, Artificial Intelligence, Computer Vision, Educational Information System, Student Attendance, Biometric Authentication, Deep Learning, Face Recognition in Education. Combining multiple keywords in this manner generally produces more focused search results.

2.8 Using Boolean Operators

Boolean Operators improve the accuracy and relevance of literature searches.

Operator	Purpose	Example
AND	Narrows the search by combining concepts	Facial Recognition AND Attendance Management
OR	Broadens the search by including synonyms	Face Recognition OR Facial Recognition
NOT	Excludes unwanted topics	Attendance System NOT Employee Attendance

Example search: ("Facial Recognition" AND Attendance Management) AND (Higher Education OR University). Using Boolean Operators helps retrieve more relevant academic publications while reducing unrelated results.

2.9 Finding Credible Sources

Not all information sources are equally reliable. Students should prioritise scholarly and peer-reviewed publications, such as:

- Journal articles.
- Conference proceedings.
- Academic books.
- Government publications.
- Industry standards.
- Technical reports from recognised organisations.

Useful academic databases include Google Scholar, IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, and Scopus (where available). General websites, blogs, discussion forums, or AI-generated content should not be used as primary academic evidence unless supported by reliable scholarly sources.

COMMON MISTAKE
Using less credible websites or blogs as the primary source of evidence instead of peer-reviewed academic literature.

Sources of Knowledge

Knowledge can be obtained from many different types of sources. Students should not rely on only one type of source when building their literature review.

Academic Sources

- Journal Articles
- Conference Proceedings
- Books
- Theses
- Systematic Reviews

Industry Sources

- Technical Documentation
- Industry Reports
- Government Publications
- Standards
- White Papers

Software Sources

- Commercial Systems
- Open-source Projects
- Mobile Applications
- Web Applications
- GitHub Repositories

Each type of source provides a different perspective that contributes to a deeper understanding of the problem.

BEST PRACTICE
Combine academic evidence with practical industry knowledge.

2.10 Evaluating the Quality of Literature

Not every published article is equally useful. When selecting literature, consider:

- Its relevance to your research problem.
- Publication date.
- The credibility of the publisher or journal.
- The research methodology used.
- The strength of evidence presented.

- Its applicability to your proposed software solution.

A small number of highly relevant studies is usually more valuable than a large collection of unrelated references.

2.11 Organising the Literature

One common mistake is organising literature paper-by-paper. Instead, literature should be organised according to research themes that progressively narrow from the broad research domains identified earlier towards the specific proposed solution, mirroring the literature review funnel introduced earlier in this chapter.

For the running example:

Digital Transformation in Higher Education
↓
Student Attendance Management
↓
Challenges in Managing Student Lecture Attendance
↓
Approaches taken by Information Systems for Attendance Management (including AI in attendance management)
↓
Biometric Authentication for Attendance Management
↓
Facial Recognition Technology
↓
Facial Recognition-Based Attendance Management Systems (Similar Systems)

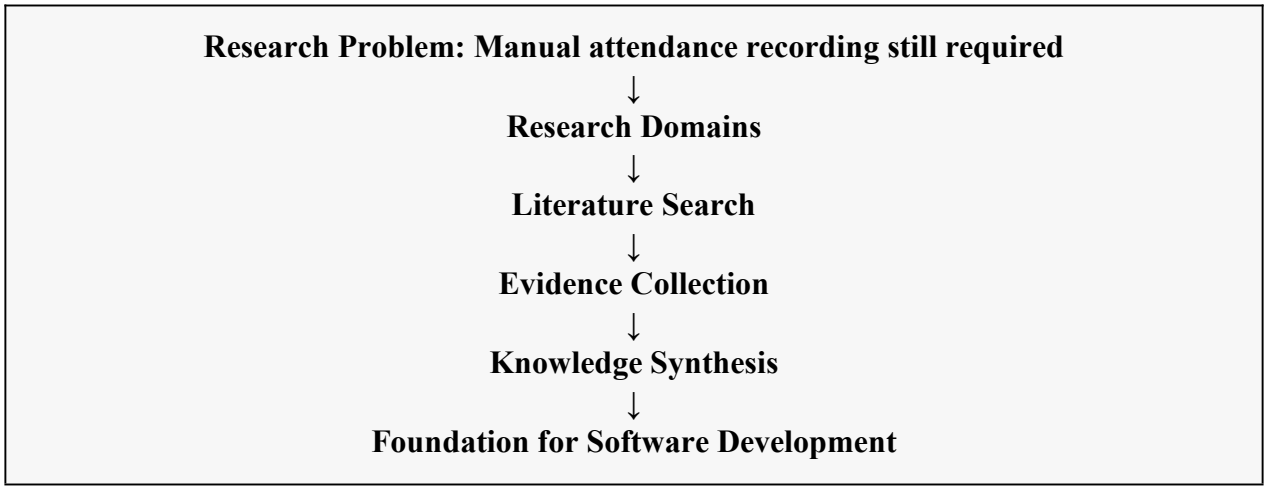
Organising literature this way, narrowing step by step rather than around a list of unordered themes, allows students to see how the evidence narrows naturally towards their proposed system, and prepares them for the critical analysis performed in Chapter 3.

BEST PRACTICE

Organise literature around concepts and themes rather than individual authors or articles. This produces a more coherent and analytical literature review.

2.12 Running Example

Using the running project, the literature review process might look like this:



This illustrates how literature supports software engineering by providing the evidence needed to understand existing knowledge before proposing a new solution.

Chapter Summary

After completing this chapter, students should understand that a literature review establishes the knowledge foundation for software development; research domains help define the scope of the literature review; effective keyword selection and Boolean Operators improve literature searches; academic databases provide credible and reliable sources of evidence; literature should be organised into themes rather than summarised individually; and a strong literature review prepares students to critically analyse existing work and identify research gaps.

Transition to Chapter 3

The literature review helps us discover what is already known about a research problem. The next step is to analyse and compare this knowledge critically. Chapter 3: Learning from Existing Knowledge and Systems explains how to synthesise literature, evaluate existing software systems, compare their strengths and limitations, and identify opportunities for improvement that justify the proposed solution.

CHAPTER 3

Learning from Existing Knowledge and Systems

Chapter Purpose

A successful software solution is rarely developed in isolation. Before proposing a new system, researchers must understand existing academic knowledge, analyse current software solutions, compare their strengths and limitations, and identify opportunities for improvement. This chapter explains how to synthesise literature, critically analyse similar systems, and transform existing knowledge into evidence that supports the proposed software solution.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the purpose of literature synthesis.
- Differentiate between summarising and synthesising literature.
- Analyse existing software systems critically.
- Compare similar systems using meaningful evaluation criteria.
- Identify strengths, limitations, and research opportunities.
- Build an evidence-based foundation for the proposed software solution.

RESEARCHER'S THINKING
Before I decided this word belonged in my project title, what did I need to understand first?

3.1 From Literature Review to Literature Synthesis

After identifying relevant literature, the next step is literature synthesis. Many students mistakenly review each paper individually without identifying connections between different studies. A good literature review does not simply describe previous work, it integrates evidence from multiple sources to develop a comprehensive understanding of the research problem.

The literature gathered in Chapter 2 already follows a broad-to-specific structure, moving from wide research domains down toward the proposed system. Synthesis in this chapter should integrate that evidence progressively, in the same order, rather than jumping directly to conclusions about the proposed solution.

Literature synthesis helps researchers to:

- Identify common findings across studies.
- Compare different approaches and technologies.
- Recognise conflicting viewpoints.
- Understand current research trends.

- Identify limitations that remain unresolved.

KEY MESSAGE
Do not ask, "What did this paper say?" Instead ask, "What do all these papers collectively tell me about my research problem?"

3.2 Summarising vs Synthesising Literature

Understanding the difference between summarising and synthesising is essential.

Summarising Literature	Synthesising Literature
Describes one article at a time.	Integrates findings from multiple studies.
Focuses on individual papers.	Focuses on themes, patterns, and relationships.
Reports what each author did.	Explains what the collective evidence suggests.
Limited critical thinking.	Demonstrates analysis, comparison, and evaluation.

Example

SUMMARY
Paper A used facial recognition for attendance. Paper B used QR codes. Paper C used fingerprint authentication.

SYNTHESIS
Existing studies demonstrate that biometric technologies improve attendance accuracy. Facial recognition offers contactless authentication and greater convenience than fingerprint systems, while QR code systems remain vulnerable to proxy attendance. Collectively, these findings suggest that AI-based facial recognition has strong potential for improving attendance management in educational institutions(Author(s) 1,Date 1;Author(s) 2, Date 2..).

BEST PRACTICE
Write about ideas and evidence, not individual papers.

3.3 Learning from Existing Software Systems

Academic literature explains theories and research findings, but software developers must also examine existing software systems. Analysing similar systems helps researchers understand existing features, implementation approaches, technologies used, usability, strengths, weaknesses, and opportunities for improvement.

The objective is not to copy an existing system, but to learn from previous solutions and identify how a new system can provide additional value.

3.4 Identifying Similar Systems

For the running example, Smart Facial Recognition-Based Attendance Management System to Improve Student Lecture Attendance at Techspire College, possible similar systems include:

- Facial Recognition Attendance Systems.
- QR Code Attendance Systems.
- RFID Attendance Systems.
- Fingerprint Attendance Systems.
- Smart Classroom Management Systems.
- University Attendance Management Platforms.

Each system provides valuable insights into current practices, implementation strategies, and remaining challenges.

3.5 Comparing Similar Systems

A comparison table helps researchers evaluate multiple systems objectively. An example comparison framework is shown below.

Evaluation Criteria	**System A**	**System B**	**System C**
Attendance Automation	Yes	Yes	Partial
Facial Recognition	Yes	No	Yes
QR Code Support	No	Yes	No
Prevents Proxy Attendance	High	Medium	Medium
Real-time Analytics	Yes	Partial	Yes
Mobile Accessibility	Yes	Yes	No
Security Features	High	Medium	Medium

Rather than simply identifying which system is “best,” students should explain why each system performs well or poorly according to the chosen evaluation criteria.

3.6 Choosing proper evaluation criteria : Analyzing Strength and Weaknesses

Critical analysis goes beyond listing features. Students should evaluate each system by asking:

- What problem does the system solve effectively?
- What limitations remain?
- Which users benefit most?
- What technologies were used?
- What challenges were reported?

- Can these limitations be addressed through a better software solution?

For example, a facial recognition attendance system may provide fast and contactless attendance recording but require high-quality cameras and sufficient lighting to achieve reliable recognition accuracy. This level of analysis demonstrates deeper understanding than simply describing system features.

Analyzing Systems from Multiple Perspectives

Many students compare software systems by listing features only, which provides little insight. Instead, students should analyse similar systems from multiple perspectives:

Functional Perspective

- What functions are provided?
- Which user requirements are addressed?
- Which important functions are missing?

User Experience Perspective

- Is the system easy to use?
- Is navigation intuitive?
- Does the interface support users effectively?

Technical Perspective

- What technologies appear to be used?
- Is the architecture suitable?
- Is the system scalable?

Security Perspective

- How are users authenticated?
- Is sensitive information protected?
- Are security best practices evident?

Performance Perspective

- Is the system responsive?
- Can it support many users?
- Are there noticeable performance issues?

Business Perspective

- Does the solution satisfy organisational needs?
- Does it improve operational efficiency?
- Does it support decision making?

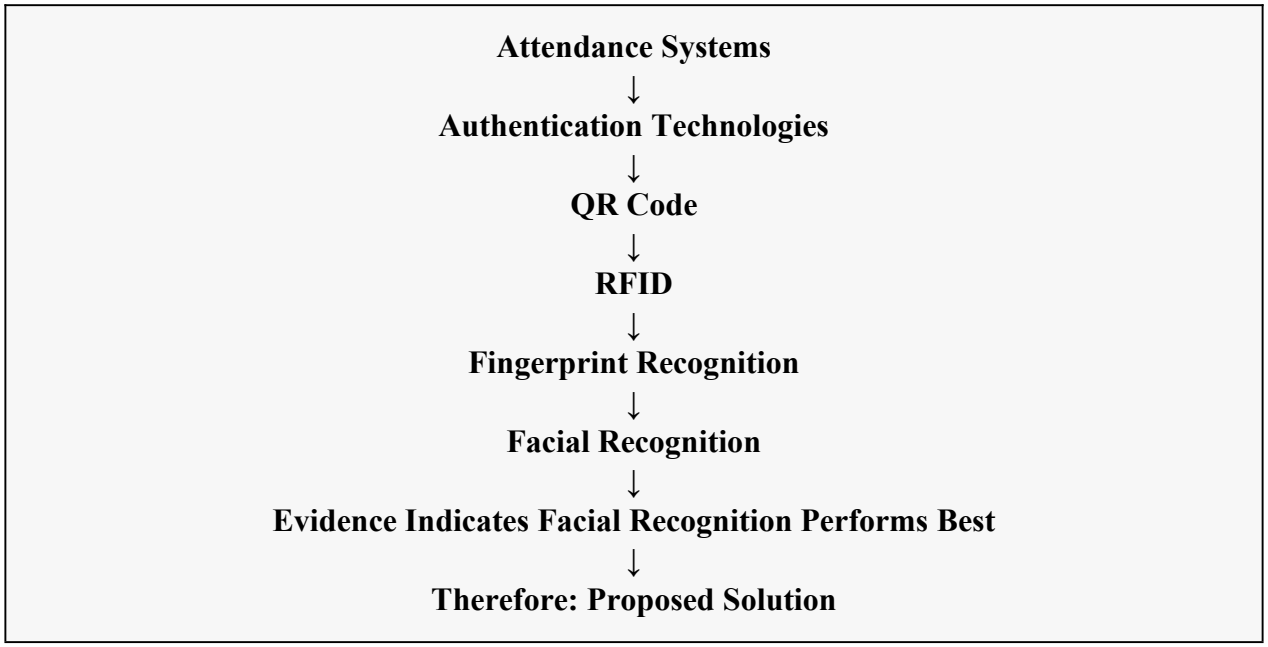
Analysing systems from multiple perspectives produces a much richer understanding than simply listing features, and strengthens the evidence used to justify the proposed system.

3.7 Building Evidence for Your Proposed System

Every observation made during the literature review and similar system analysis should contribute to the proposed solution. For the running example, research findings indicate that AI improves attendance automation, facial recognition reduces manual interaction, existing systems still experience proxy attendance, analytics remain limited in many current solutions, and educational institutions increasingly require intelligent attendance monitoring.

These findings collectively justify developing an AI-powered facial recognition attendance management system with automated attendance recording, enhanced analytics, and integration with the existing eDigital Attendance System.

The proposed solution should never be selected first and then supported with evidence. Instead, it should gradually emerge as the literature is reviewed, as illustrated below.



KEY MESSAGE

Your proposed system should emerge naturally from the evidence presented in your literature review.

3.8 Running Example

The research journey for the running project can be visualised as follows:

Research Domains
↓
Academic Literature
↓
Literature Synthesis
↓
Similar Systems
↓
Critical Comparison
↓
Strengths & Limitations
↓
Evidence-Based Insights
↓
Foundation for Proposed System

This process demonstrates that the proposed solution is supported by evidence rather than personal assumptions.

3.9 Preparing for Research Gap Identification

By this stage, students should have developed a comprehensive understanding of existing research, current technologies, commercial software, strengths of existing solutions, and unresolved limitations. The next logical step is to determine what is still missing. These unanswered questions become research opportunities that justify the development of a new software solution.

Common Mistakes

- Describing one paper after another without comparison.
- Listing software features without analysing their strengths or weaknesses.
- Comparing systems using inconsistent criteria.
- Copying comparison tables directly from published papers.
- Concluding that a proposed system is better without supporting evidence.

Best Practice

When analysing literature or similar systems, always ask:

- What does this teach me?
- What works well?
- What limitations remain?
- How does this influence my proposed system?

These questions help transform information into evidence that supports software development.

Chapter Summary

After completing this chapter, students should understand that literature synthesis combines evidence from multiple studies to develop a deeper understanding of the research problem; similar system analysis evaluates existing software solutions objectively; critical comparison identifies strengths, limitations, and opportunities for improvement; evidence gathered from literature and existing systems provides the justification for the proposed software solution; and the next stage is to identify research gaps that explain why a new solution is still required.

Transition to Chapter 4

The literature review and similar system analysis reveal what researchers and developers have already accomplished. However, no existing solution is perfect. Chapter 4: Finding the Research Gap explains how to identify unresolved problems, justify the need for the proposed system, and demonstrate the originality and contribution of the project.

CHAPTER 4

Finding the Research Gap

Chapter Purpose

A research gap is the bridge between understanding existing knowledge and proposing a new software solution. This chapter explains how to identify unresolved problems, limitations, and opportunities from the literature review and similar system analysis. Students will learn how to justify the need for their proposed system by demonstrating that existing research or software solutions do not fully address the identified problem.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the concept of a research gap.
- Identify research gaps using evidence from literature and similar systems.
- Differentiate between different types of research gaps.
- Justify why a new software solution is required.
- Demonstrate the originality and potential contribution of the proposed system.

RESEARCHER'S THINKING
Before I decided this word belonged in my project title, what did I need to understand first?

4.1 What is a Research Gap?

A research gap is an unresolved problem, limitation, or opportunity that has not been adequately addressed by existing research or software solutions. A research gap does not mean that previous researchers failed. Instead, it reflects the reality that technology, user needs, and organisational requirements continue to evolve, creating new opportunities for improvement and innovation.

In Computing and Technology, identifying a research gap provides the justification for developing a new or improved software system.

KEY MESSAGE
A research gap answers the question: "Why is another software solution still needed?"

4.2 Where Do Research Gaps Come From?

Research gaps should not be based on personal opinions or assumptions. They should emerge naturally from evidence gathered during the literature review and similar system analysis.



This ensures that the proposed system is justified by evidence rather than unsupported claims.

Because the literature review in Chapter 2 and the synthesis in Chapter 3 already move progressively from broad research domains toward the proposed system, the research gaps identified here emerge naturally from that narrowing evidence rather than being assumed in advance.

4.3 Types of Research Gaps in Computing and Technology

Research gaps may arise in different forms. Understanding these categories helps students identify meaningful opportunities for software development.

Gap Type	Description	Example
Functional Gap	Existing systems lack important features or capabilities.	Current attendance systems do not automatically recognise students using facial recognition.
Usability Gap	Systems are difficult to use or reduce user efficiency.	Lecturers must manually mark attendance despite using a digital system.
Technology Gap	New technologies have not been fully utilised.	AI and computer vision have not been effectively integrated into the current attendance process.
Performance Gap	Existing systems do not meet expected efficiency, speed, or accuracy.	Attendance recording remains time-consuming and susceptible to proxy attendance.
Security Gap	Existing solutions provide inadequate protection for data or authentication.	Weak authentication mechanisms increase the risk of attendance fraud.
Context Gap	Existing solutions have not been applied effectively within a specific environment or organisation.	Few studies focus on improving attendance management in Techspire College's existing digital workflow.

Students should remember that a single project may address multiple research gaps simultaneously.

4.4 Identifying Research Gaps from Literature

The literature review provides valuable evidence about current knowledge, existing technologies, and previous findings. When reviewing literature, ask:

- What problems continue to appear across multiple studies?
- What limitations do researchers frequently mention?
- Which recommendations for future work are repeatedly suggested?
- What technologies remain underexplored?
- Are there conflicting findings that require further investigation?

Recurring limitations often indicate opportunities for further research and software improvement.

4.5 Identifying Research Gaps from Similar Systems

Similar system analysis reveals how existing software performs in real-world environments. When evaluating existing systems, consider:

- Which user needs remain unmet?
- Which features are missing?
- Which processes still require manual intervention?
- Which technologies could improve the system?
- Which operational challenges continue to affect users?

These observations help transform software limitations into opportunities for innovation.

4.6 Running Example

Using the running project , Techspire College already operates an eDigital Attendance System. Evidence collected from literature and similar systems suggests:

- Attendance records are stored digitally.
- Lecturers still manually mark attendance for every student.
- Manual interaction interrupts teaching.
- Proxy attendance remains possible.
- Existing systems provide limited automation.
- Attendance analytics and reporting can be improved.

From this evidence, several research gaps emerge:

Functional Gap

The existing system does not provide automatic attendance recording using facial recognition.

Usability Gap

Manual attendance marking increases lecturer workload and reduces teaching efficiency.

Technology Gap

Artificial Intelligence and Computer Vision have not been integrated into the existing attendance process.

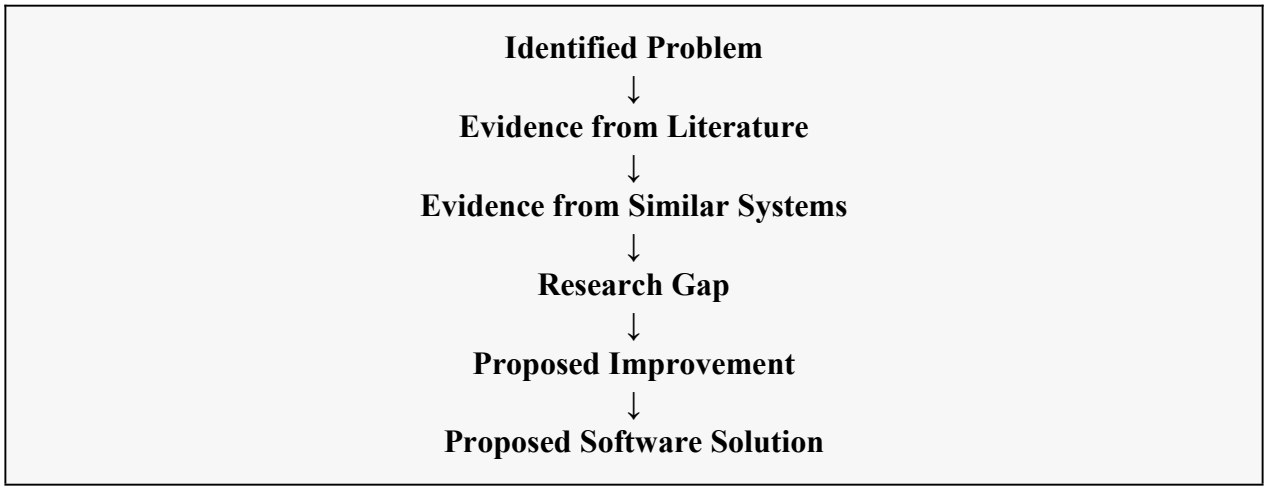
Performance Gap

Attendance recording remains slower and less efficient than an automated approach.

These gaps collectively justify the development of a Smart Facial Recognition-Based Attendance Management System that enhances the current digital attendance process rather than replacing it.

4.7 Justifying the Proposed Solution

Once research gaps have been identified, they should be directly linked to the proposed system.



BEST PRACTICE

Every feature in your proposed system should solve an identified research gap supported by evidence.

4.8 Demonstrating Research Contribution

A good proposal should clearly explain how it contributes to existing knowledge or practice. Based on the assignment requirements, a project may contribute by proposing a new solution to a new problem, proposing an improved solution to an existing problem, or applying an existing solution to a new context.

For most undergraduate software development projects, the contribution is typically an improvement to an existing system through better functionality, improved usability, enhanced

automation, stronger security, or more effective application of technology. Students should avoid making unrealistic claims such as developing a completely new technology unless supported by strong evidence.

Common Mistakes

- Claiming there is a research gap without providing supporting evidence.
- Treating personal opinions as research gaps.
- Describing system limitations without explaining why they matter.
- Proposing features that do not address any identified gap.

Best Practice

When identifying research gaps, always ask:

- What evidence supports this gap?
- Why does this limitation matter?
- How does it affect users?
- How will my proposed system address it?
- Does every proposed feature solve a real problem?

These questions help ensure that the proposed system is evidence-based and academically justified.

Chapter Summary

After completing this chapter, students should understand that research gaps emerge from evidence collected through literature review and similar system analysis; different types of research gaps provide opportunities for software improvement and innovation; every proposed feature should address an identified research gap; a well-justified research gap demonstrates the need, originality, and contribution of the proposed software solution; and identifying research gaps provides the foundation for making informed technical decisions in the next stage of the proposal.

Transition to Chapter 5

After identifying research gaps, the next challenge is deciding how to solve them. Software developers must select appropriate technologies, architectures, databases, frameworks, and security mechanisms based on evidence rather than assumptions. Chapter 5: Making Evidence-Based Technical Decisions explains how research supports these important software engineering decisions.

CHAPTER 5

Making Evidence-Based Technical Decisions

Chapter Purpose

Developing a software solution involves making numerous technical decisions, including selecting programming languages, frameworks, databases, architectures, security mechanisms, and deployment platforms. These decisions should not be based solely on personal preference or familiarity. Instead, they should be supported by evidence gathered through literature review, similar system analysis, user needs, and project requirements. This chapter explains how research informs technical decision making and helps students justify the technologies and approaches proposed for their software project.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the importance of evidence-based technical decision making.
- Identify common technical decisions made during software development.
- Evaluate alternative technologies using appropriate criteria.
- Justify technical choices using evidence from research and project requirements.
- Demonstrate how research supports software engineering decisions.

RESEARCHER'S THINKING
Before I decided this belonged in my project title, what did I need to understand first?

5.1 Why Technical Decisions Matter

Every software project requires numerous technical decisions that influence the quality, usability, security, performance, and maintainability of the final system, for example:

- Which programming language should be used.
- Which framework best supports the project requirements.
- Which database is most appropriate.
- Which software architecture should be adopted.
- Which authentication mechanism provides adequate security.
- Whether the system should be deployed on-premises or in the cloud.

Poor technical decisions may lead to inefficient development, security vulnerabilities, poor system performance, or software that fails to meet user needs.

KEY MESSAGE

In Computing and Technology, technical decisions should be based on evidence, not assumptions, trends, or personal preference.

5.2 Research as the Foundation for Technical Decisions

Research provides the evidence needed to make informed software engineering decisions. Evidence may come from academic literature, similar software systems, user requirements, industry standards and best practices, organisational requirements, technical documentation, and performance evaluations and benchmarking studies.

Rather than selecting technologies because they are popular or familiar, students should explain why they are appropriate for the specific problem being solved.

✗ WRONG

Facial Recognition was selected because it is modern.

✓ CORRECT

Literature comparing RFID, QR Code, Fingerprint Recognition and Facial Recognition indicates that facial recognition provides contactless authentication, reduces proxy attendance, and integrates effectively with intelligent attendance systems. Therefore facial recognition is selected for the proposed system.

Notice that the correct justification is drawn directly from the literature. The literature chooses the technology; the student does not choose it based on personal preference.

5.3 Common Technical Decisions in Software Projects

Software developers commonly make decisions in several key areas.

Technical Area	Example Decisions
Programming Language	Java, Python, C#, JavaScript
Framework	Laravel, Spring Boot, React, Flutter
Database	MySQL, PostgreSQL, MongoDB, Firebase
Software Architecture	Three-tier, MVC, Microservices
Authentication	Username & Password, Biometric, OAuth, Multi-factor Authentication
Deployment Platform	Cloud, On-premises, Hybrid
AI / Machine Learning	TensorFlow, OpenCV, FaceNet, YOLO
Security	Encryption, Access Control, Secure Communication

Each decision should be justified according to project requirements rather than personal familiarity.

5.4 Evaluating Technology Alternatives

Selecting the most suitable technology requires comparing available alternatives using objective evaluation criteria, such as:

- Functional suitability.
- Performance.
- Scalability.
- Security.
- Ease of development.
- Maintainability.
- Compatibility with existing systems.
- Cost and licensing.
- Community support and documentation.

Example Comparison

Criteria	OpenCV	Commercial Face Recognition API
Cost	Free and open source	Subscription or usage fees
Customisation	High	Limited
Integration	Flexible	API dependent
Community Support	Extensive	Vendor specific
Suitability for Academic Project	High	Moderate

The objective is not to identify a universally “best” technology, but the technology that best fits the project's requirements.

5.5 Running Example

Using the Smart Facial Recognition-Based Attendance Management System, research findings indicate that facial recognition provides contactless authentication, AI improves attendance automation, existing digital attendance systems already manage attendance records, lecturers require minimal interruption during teaching, and real-time reporting is desirable for academic monitoring.

Technical Decision	Evidence-Based Justification
Facial Recognition	Reduces manual attendance recording and helps prevent proxy attendance.

Technical Decision	Evidence-Based Justification
AI-based Image Processing	Enables automatic student recognition.
MySQL Database	Suitable for structured attendance records and reporting.
Web-based Application	Accessible across multiple devices within the institution.
Integration with Existing eDigital Attendance System	Preserves current workflows while enhancing automation.

Notice that every technical choice is linked directly to evidence rather than personal preference.

5.6 Aligning Technical Decisions with User Requirements

Technical decisions should always support user needs.

User Requirement: Lecturers want attendance recording to take place automatically without interrupting teaching.

↓

Technical Decision: Implement AI-based facial recognition.

↓

Expected Benefit: Attendance is recorded automatically while reducing lecturer workload.

This alignment ensures that technology choices directly contribute to solving real-world problems.

5.7 Justifying Technical Decisions

When documenting technical decisions in a proposal, students should explain: (1) what decision was made, (2) why this option was selected, (3) what evidence supports the decision, and (4) how it helps solve the identified problem.

EXAMPLE

Decision: A web-based application will be developed. Justification: A web-based solution provides platform independence, centralised deployment, and easier access for lecturers and administrators. It also integrates effectively with the institution's existing digital infrastructure.

This structured approach demonstrates critical thinking and strengthens the academic quality of the proposal.

BEST PRACTICE

Every major technical decision should be supported by research evidence and clearly linked to project requirements.

5.8 From Research to Software Development Decisions



This process demonstrates that software development decisions are informed by research rather than assumptions.

5.9 Preparing for Research Methodology

By this stage, students have understood the research problem, reviewed existing knowledge, analysed similar systems, identified research gaps, and justified the proposed technical solution. The next step is to plan how the research and software development will be conducted systematically. This is achieved through the Research Methodology.

Common Mistakes

- Selecting technologies because they are familiar.
- Choosing frameworks without comparing alternatives.
- Listing programming languages without justification.
- Copying technology choices from previous projects.
- Ignoring user requirements when selecting technical solutions.

Best Practice

Before making any major technical decision, ask:

- What problem does this decision solve?
- What evidence supports this choice?
- Is it appropriate for the project scope?

- Can I justify this decision academically?
- Does it align with user requirements?

If you cannot answer these questions, the decision requires further investigation.

Chapter Summary

After completing this chapter, students should understand that software development involves many important technical decisions; technical decisions should be based on research evidence rather than assumptions or personal preference; alternative technologies should be evaluated using objective criteria; every technical choice should address identified user needs and research gaps; and evidence-based decision making provides a strong foundation for the Research Methodology and the successful development of the proposed software solution.

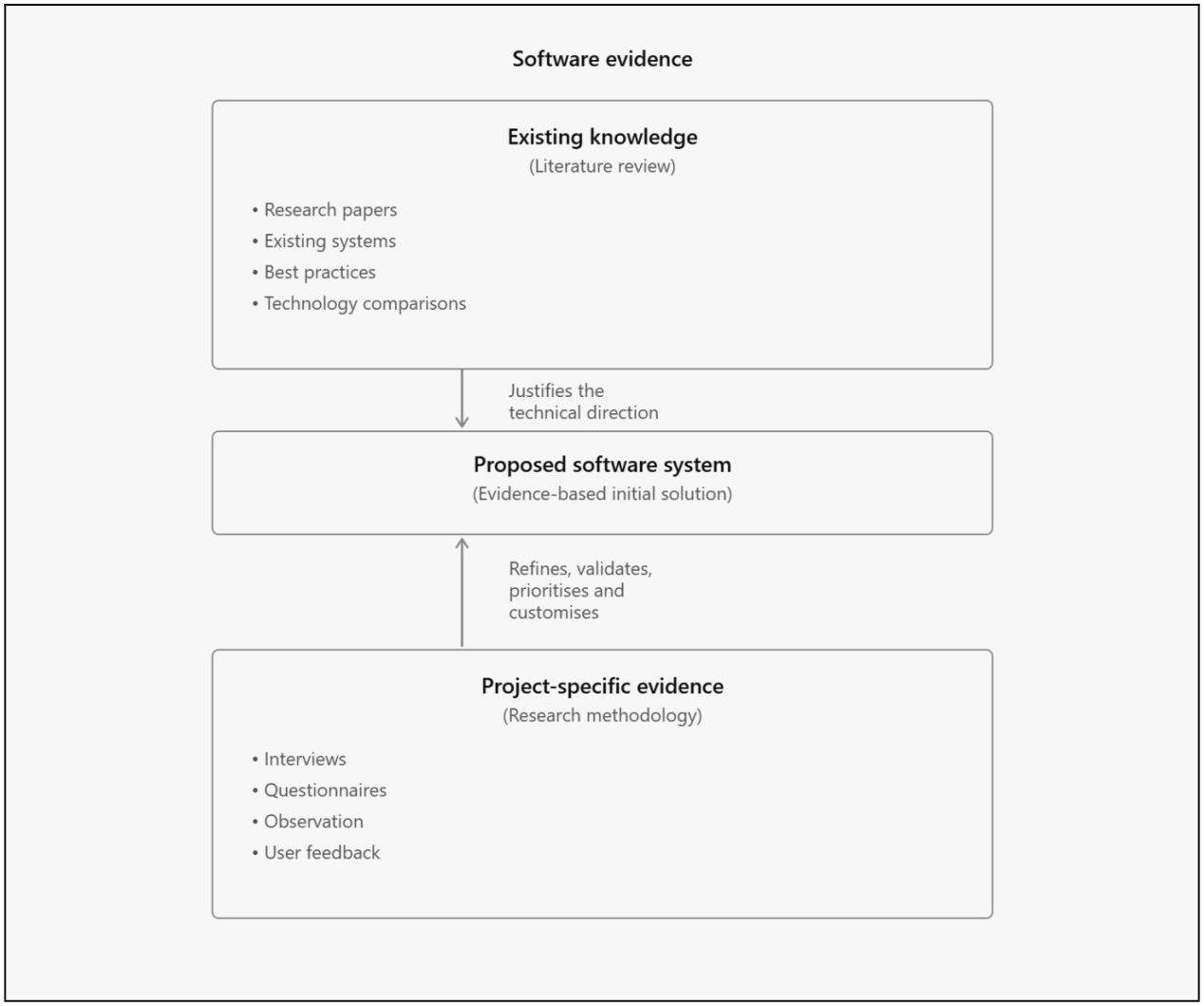
Transition to Chapter 6

After determining what technologies and approaches are appropriate for the project, the next step is to plan how the research and software development will be carried out. Chapter 6: Planning the Research Methodology explains how to design a systematic research process that supports every stage of software development, from selecting a software development methodology to collecting evidence, engaging users, and ensuring ethical research practices.

The Literature Review has now answered one essential question: why is this the appropriate solution? However, justification alone does not produce working software. The next step is discovering how this justified solution fits the organisation, how it will be developed, and how it will be evaluated. This is the purpose of Research Methodology.

The Most Important Diagram in This Guide

This diagram is the conceptual bridge between everything covered so far in the Literature Review chapters and everything covered from this point forward in the Research Methodology chapters. Every later chapter connects back to it.


Software evidence
Existing knowledge
(Literature review)
• Research papers
• Existing systems
• Best practices
• Technology comparisons
Justifies the technical direction
Proposed software system
(Evidence-based initial solution)
Refines, validates, prioritises and customises
Project-specific evidence
(Research methodology)
• Interviews
• Questionnaires
• Observation
• User feedback

Existing Knowledge vs. Project-Specific Knowledge

Students often confuse these two, so it is worth stating the difference plainly:

	Existing Knowledge (Literature Review)	**Project-Specific Knowledge (Research Methodology)**
Obtained from	Journals, books, previous systems, previous researchers	Interviews, questionnaires, observations, client meetings, usability testing
Purpose	Understand what generally works	Understand what THIS organisation(Specific Context) needs
Produces	Evidence-based technical decisions	Organisation(Specific Context)-specific requirements

KEY MESSAGE
Literature tells us what generally works. Requirements Gathering tells us how that solution should work for this specific organisation.

Potential Conceptual Gap:

Students naturally ask: **"If the Literature Review already justifies every technical decision, what exactly is left for Research Methodology?"** This deserves a direct answer.

- The Literature Review answers: **why is this the appropriate solution?**
- Research Methodology answers: **how will this justified solution be adapted, developed, validated, and evaluated?**

One justifies. The other operationalises. **Both are research, they simply answer different questions at different points in the project. And one supports the other.**

Chapter 6: Planning the Research Methodology explains how to design a systematic research process that supports every stage of software development, from selecting a software development methodology to collecting evidence, engaging users, and ensuring ethical research practices.

CHAPTER 6

Planning the Research Methodology

Chapter Purpose

Up to this point, you have used academic evidence to justify your proposed software solution. You now understand what software should be developed and why it is the most appropriate solution. The next stage is planning how that justified solution will be transformed into a working software system. This begins by selecting an appropriate Software Development Methodology, which provides the overall roadmap for developing, managing, and evaluating the proposed system. Research activities are then integrated into this development process to ensure that each stage is supported by appropriate evidence and user feedback.

A Research Methodology provides this roadmap. It explains how evidence will be collected, how users will be involved, how technical decisions will be justified, and how the proposed software solution will be developed and evaluated. Rather than treating research as a separate activity, the methodology demonstrates how research continuously supports every stage of software development.

This chapter explains each component of the Research Methodology and demonstrates how they work together to produce an organised, evidence-based software development process.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the purpose of a Research Methodology in Computing and Technology.
- Distinguish between Software Development Methodology, Research Design, Research Approach, and Research Methods.
- Select and justify an appropriate Software Development Methodology for a software project.
- Plan a research process that supports software development.
- Identify suitable target users, sampling strategies, and data collection methods.
- Explain the importance of ethical considerations when conducting research involving human participants.

RESEARCHER'S THINKING

Now that I know what I am building and why, how should I plan,validate, develop, and evaluate it?

6.1 Why Research Methodology Matters

Before developing any software system, researchers must first determine how the entire project will be conducted. This includes planning how software will be developed how evidence will be collected, how users will participate, and how the final system will be evaluated.

A well-designed Research Methodology provides this overall plan. It ensures that every stage of the project follows a logical, systematic, and evidence-based process rather than relying on assumptions or personal opinions.

In Computing and Technology, the Research Methodology does much more than explain research methods. It demonstrates how research supports software development by helping researchers understand the real-world problem, identify user requirements, justify technical decisions, validate software features, evaluate the completed system, and recommend future improvements.

Without a clear methodology, software development may become disorganised, resulting in poor requirements, weak technical decisions, and software that fails to meet user needs.

Research and Software Development



This illustrates that research is not performed only at the beginning of the project. Instead, it continuously informs and supports software development from planning through evaluation.

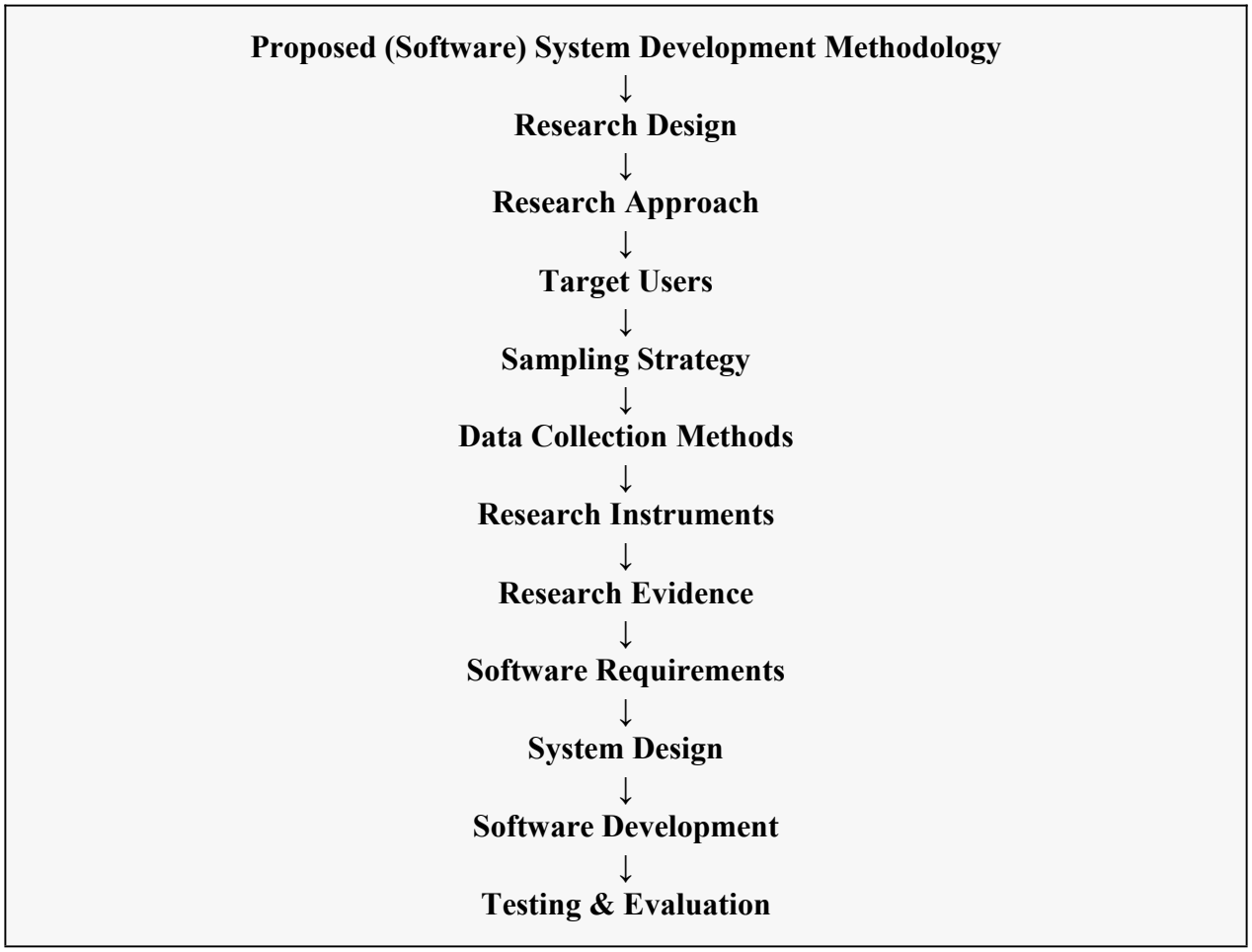
KEY MESSAGE

A Research Methodology is the overall plan that explains how research and software development will work together to achieve the project objectives.

6.2 Understanding the Methodology Hierarchy

Students often confuse terms such as Software Development Methodology, Research Design, Research Approach, Research Methods, and Research Instruments because they appear closely related. However, each component serves a different purpose within the research process.

The following hierarchy demonstrates how these components fit together.



Rather than viewing these as independent report sections, students should understand them as one continuous process that transforms research evidence into a successful software solution. The table below summarises the purpose of each component.

Component	Main Question Answered	Purpose
Proposed System Development Methodology	How will the software be developed?	Provides the framework for planning, designing, developing, testing, and delivering the software.
Research Design	How will research support software development?	Explains how research activities are organised throughout the project.
Research Approach	What type of evidence is required?	Determines whether qualitative, quantitative, or mixed methods will be

Component	Main Question Answered	Purpose
		used.
Target Users	Who can provide the required information?	Identifies the people who will use or contribute to the proposed system.
Sampling Strategy	Which participants will take part in the research?	Explains how participants are selected from the target users.
Data Collection Methods	How will evidence be collected?	Identifies the methods used to gather research data.
Research Instruments	Which tools will collect the evidence?	Specifies the instruments used during data collection, such as questionnaires or interview guides.

WHY SYSTEM DEVELOPMENT METHODOLOGY COMES FIRST

This guide intentionally does not keep Research Design at the beginning. Students first decide how the software will be developed, and only afterwards decide what research activities are required throughout that development process. The Software Development Methodology is therefore the backbone of the project, and Research Design supports that backbone rather than replacing it.

Running Example

For the Smart Facial Recognition-Based Attendance Management System:

- Software Development Methodology: Agile
- Research Design: Research activities aligned with Agile iterations.
- Research Approach: Mixed Methods.
- Target Users: Lecturers, students, and academic administration staff.
- Sampling Strategy: Purposive Sampling.
- Data Collection Methods: Questionnaires, interviews, observations, and document analysis.
- Research Instruments: Google Forms questionnaire, interview guide, and observation checklist.

Notice how each component builds upon the previous one to create a complete methodology.

BEST PRACTICE

Think of the methodology as a roadmap. Every component has a specific purpose, but together they explain how the project will be conducted systematically.

Official Structure of the Research Methodology Section in the Report

When writing the proposal, the Research Methodology section (Section 3 of the report) should be organised using the following subsections:

- 3 Research Methodology
- 3.1 Proposed System Development Methodology
- 3.2 Research Design
- 3.3 Research Approach
- 3.4 Target Users
- 3.5 Sampling Strategy
- 3.6 Data Collection Methods
- 3.7 Ethical Considerations

Each subsection contributes to explaining how the project will be conducted, and the remainder of this chapter addresses each of these subsections in turn.

STUDENT QUESTION
How will I validate, build, and evaluate this software?

6.3 Proposed System Development Methodology

Why Software Development Needs a Methodology

Before a specific methodology is chosen, it is worth understanding why one is needed at all. Software development is not a single activity, but a set of coordinated activities such as gathering requirements, designing the system, writing code, testing, and deploying the solution. Without a structured process to organise these activities, projects tend to run into predictable problems.

- Work becomes disorganised, with team members unclear on priorities.
- Requirements are missed or misunderstood.
- Testing is delayed until problems are difficult and costly to fix.
- Projects overrun, or fail to deliver a working system.

A Software Development Methodology addresses this by providing a structured process that defines what activities take place, in what order, how the project progresses, and how the team coordinates its work. Choosing a methodology is therefore itself an evidence-based decision: it should be justified by the characteristics of the project, in the same way every other decision in this guide is justified by evidence rather than assumption.

KEY MESSAGE
A Software Development Methodology exists to organise the many coordinated activities of software development into a structured, manageable process.

What is a System Development Methodology?

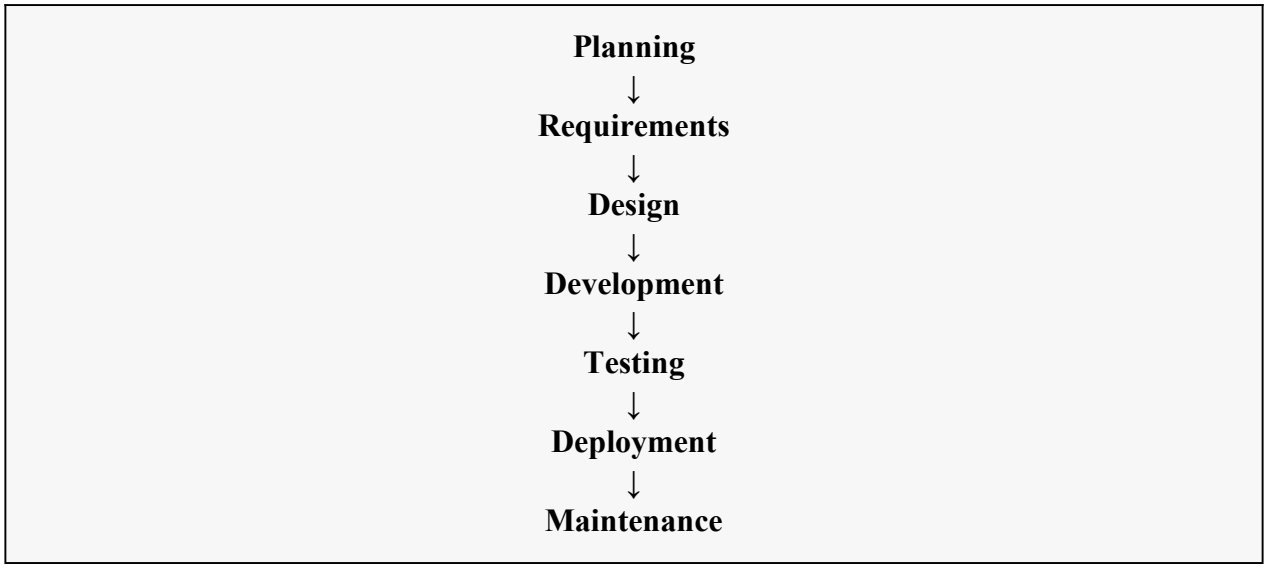
A System Development Methodology is a structured framework that explains how the software system will be planned, designed, developed, tested, and deployed. It provides a clear development process that helps software developers organize project activities, manage resources, monitor progress, and deliver a high-quality software solution.

Students should remember that a Software Development Methodology is different from a Research Methodology. A Software Development Methodology explains how the software will be built, while a Research Methodology explains how the research and software development will be conducted together. So, Software Development Methodology is just a part of overall Research Methodology.

SDLC vs SDM: Two Different Questions

Two terms are frequently confused at this stage: the Software Development Life Cycle (SDLC) and the Software Development Methodology (SDM). Although related, they answer two different questions.

Term	Question It Answers	Focus
Software Development Life Cycle (SDLC)	What phases does software development go through?	The universal set of phases every software project passes through.
Software Development Methodology (SDM)	How should we move through those phases?	The approach used to plan, order, and manage those phases.



These phases remain broadly the same across projects. What changes between projects is the methodology used to move through them, which is the subject of the remainder of this section.

KEY MESSAGE
SDLC describes WHAT phases exist. SDM describes HOW a project moves through those phases.

The Same SDLC, Organised Differently

Every methodology still moves through the SDLC phases described above. What differs is how each methodology organises or manages those phases.

Methodology	How It Organises the SDLC Phases
Waterfall	Performs the SDLC phases sequentially, completing one before starting the next.
Agile	Performs the SDLC phases iteratively, repeating short cycles to deliver working increments.
Prototype	Revisits the requirements and design phases repeatedly through working prototypes.
Spiral	Repeats the SDLC phases around a cycle of risk analysis and refinement.
Rational Unified Process (RUP)	Progresses through the SDLC phases iteratively, with strong architectural planning early on.
Rapid Application Development (RAD)	Compresses the design and development phases to emphasise rapid delivery.

The Software Development Methodology Landscape

Before individual methodologies are discussed in detail, it is useful to see how they relate to one another. The landscape below groups common methodologies according to their underlying approach.

Category	Example Methodologies
Plan-Driven / Predictive	Waterfall, V-Model
Risk-Driven	Spiral
Prototype-Based	Prototyping
Rapid Development	Rapid Application Development (RAD)
Iterative & Incremental	Rational Unified Process (RUP)
Agile Approaches	Scrum, Kanban, Extreme Programming (XP), Crystal, Feature Driven Development (FDD), Dynamic Systems Development Method (DSDM), Lean Software Development

The objective is not to master every methodology listed here. The objective is to recognise how they relate to one another, so that Agile approaches, for example, are not mistaken for the only alternative to Waterfall.

Understanding Agile as a Philosophy

A common misunderstanding is treating Agile and Scrum as the same thing. Agile is not one specific methodology. Agile is a philosophy, built around a shared set of values and principles that favour flexibility, collaboration, and continuous improvement.

Scrum, Kanban, Extreme Programming (XP), Crystal, Feature Driven Development (FDD), Dynamic Systems Development Method (DSDM), and Lean Software Development are different Agile approaches. Each implements the same underlying Agile principles in a different way.

Agile Philosophy
↓
Scrum, Kanban, XP, Crystal, FDD, DSDM, Lean Software Development

KEY MESSAGE
Agile is a philosophy, not a single methodology. Scrum is one of several ways of putting Agile principles into practice.

To avoid this confusion, it helps to see exactly where Agile sits within the wider set of software development methodologies, and how it relates to specific Agile frameworks.

Software Development Methodologies
↓
Agile Philosophy
↓
Agile Frameworks / Approaches
↓
Scrum
↓
Kanban
↓
Extreme Programming (XP)
↓
Crystal
↓
Feature-Driven Development (FDD)
↓
Dynamic Systems Development Method (DSDM)
↓
Lean Software Development

Agile itself only defines values and principles. Scrum, Kanban, XP, DSDM, and the other approaches below provide practical ways of implementing those principles. This is why Agile and Scrum are not interchangeable terms.

Agile Approach	Main Focus	Best Suited For	Key Characteristics
Scrum	Iterative delivery through fixed-length sprints	Projects with a defined backlog and regular stakeholder feedback	Roles (Product Owner, Scrum Master, Developers), Sprint planning, Daily stand-ups, Sprint reviews, Product Backlog, Sprint Backlog, Sprint Review, retrospectives
Kanban	Continuous flow of work visualised on a board	Projects with continuous incoming work and maintenance	Visual Kanban board, Work-In-Progress limits, Continuous delivery, No fixed sprints
Extreme Programming (XP)	High software quality through best engineering practices	Small to Medium Projects where technical quality and frequent releases matter most	Pair programming(Driver & Navigator), Test-Driven Development (TDD), Continuous Integration, Frequent releases
Feature-Driven Development (FDD)	Development organised around specific features, Build software feature by feature	Larger teams needing feature-based progress tracking, Large object-oriented systems	Domain modelling, Feature lists, Short iterative feature implementation
Dynamic Systems Development Method (DSDM)	Business-driven delivery within a fixed timeframe	Projects with strict deadlines and changing requirements	Timeboxing, Active user involvement, Frequent delivery, Strong project governance
Crystal	Lightweight, people-focused development	Small teams prioritising communication over process	Minimal formal process, frequent delivery, reflection
Lean Software Development	Eliminating waste and maximising value	Projects seeking efficiency and continuous improvement	Remove waste, Fast delivery, Continuous improvement, Respect for people

These examples confirm that Scrum is only one of several ways of applying Agile values in practice; no single approach below is equivalent to Agile as a whole.

Concept	Description	Examples
Software Development Methodology	Overall approach for organising software development	Waterfall, Spiral, RUP, Agile
Agile Framework / Approach	A specific implementation of Agile principles	Scrum, Kanban, XP
Agile Practices	Individual techniques used within Agile frameworks	Sprint Planning, Daily Stand-up, Pair Programming, Sprint Review, Retrospectives

GUIDANCE

Although multiple Agile approaches exist, most undergraduate Computing and Technology projects are well suited to Scrum, since it offers simple project planning, iterative development, continuous stakeholder feedback, manageable sprint-based progress, and straightforward documentation. This is offered as guidance rather than a requirement; another Agile approach may be justified if the evidence supports it.

Rational Unified Process (RUP)

Iterative development did not begin with Agile. The Rational Unified Process (RUP) is an earlier methodology that is:

- Iterative and incremental, developing the system through repeated cycles.
- Architecture-centric, establishing the software architecture early.
- Use-case driven, organising requirements and development around use cases.
- Risk-aware, addressing the highest-risk elements first.
- More structured than Scrum, with defined phases and formal artefacts.

RUP shares characteristics with Agile approaches, such as iteration, but it is not itself an Agile framework. Iterative and incremental development did not originate with Agile: RUP applies these same principles while placing stronger emphasis on architecture, documentation, and structured project governance, which is why it appears separately within the methodology landscape.

Characteristics versus Methodologies

Students sometimes describe a methodology using a word that is actually a characteristic rather than the name of an approach. The two should not be confused.

Characteristics (describe HOW a methodology behaves)	**Methodologies / Approaches (the actual names)**
iterative, incremental, predictive, adaptive, risk-driven, rapid	Waterfall, Spiral, Prototype, RAD, RUP, Scrum, XP, Kanban

For example, describing an approach as “iterative” explains a characteristic it has, whereas Scrum is the name of an actual methodology that happens to have that characteristic.

Common Software Development Methodologies

Methodology	Best Used When	Strengths	Example Project
Agile	Requirements may evolve or user feedback is required throughout development.	Flexible, iterative, and encourages continuous improvement.	Smart Attendance Management System
Waterfall	Requirements are stable and clearly defined from the beginning.	Well-structured and easy to manage.	Payroll Management System
Rapid Application Development (RAD)	Rapid prototyping and frequent user feedback are required.	Fast development and early prototype delivery.	Event Registration System
Spiral	Projects involve significant technical or security risks.	Continuous risk assessment and refinement.	Online Banking System
Prototype Model	User requirements are not fully understood at the beginning.	Allows early validation of system requirements.	Healthcare Appointment System
Rational Unified Process (RUP)	Enterprise projects need strong architectural planning and risk management.	Iterative, incremental, use-case driven, architecture-centric, and risk-aware.	Enterprise Resource Planning System

Students are not expected to choose the most popular methodology. Instead, they should select the methodology that best matches the characteristics of their proposed project.

Choosing an Appropriate Methodology

When selecting a Software Development Methodology, students should consider:

- Are the project requirements likely to change?
- Will users provide feedback during development?
- Is the project technically complex?
- Is rapid prototyping required?
- What level of project risk is involved?

The selected methodology should always support the project's objectives and development requirements.

The characteristics below offer a quick-reference guide for methodology selection, based on the same evidence-based reasoning used throughout this guide.

Project Characteristic	Suggested Methodology
Stable, clearly defined requirements	Waterfall
Requirements likely to change	Agile
High project risk	Spiral
Need for early user feedback on requirements	Prototype
Rapid delivery is a priority	RAD

Project Characteristic	Suggested Methodology
Architecture-heavy enterprise project	RUP

What Should Students Write?

In this subsection of the proposal, students should introduce the selected Software Development Methodology, briefly explain its main phases or principles, justify why it is suitable for the proposed project, and explain how it supports the successful development of the software system. The emphasis should be on justification, not simply describing the methodology.

Running Example

Project: Smart Facial Recognition-Based Attendance Management System.

The reasoning chain below shows how this decision follows from evidence already gathered earlier in the proposal, rather than from familiarity or personal preference.

Existing eDigital Attendance System already in place
↓
Requirements likely to evolve as facial recognition is refined
↓
Lecturer feedback needed on system usability
↓
Student feedback needed on accuracy and experience
↓
Facial recognition accuracy improves iteratively through testing
↓
Agile philosophy selected
↓
Implemented using Scrum
↓
Incremental software development and delivery

Selected Methodology: Agile Software Development.

Justification: Agile is selected because the proposed system involves AI-based facial recognition, user interface design, and integration with the existing eDigital Attendance System. These components are likely to require continuous testing, refinement, and user feedback throughout development. Agile's iterative approach enables new features to be developed incrementally while allowing improvements based on feedback from lecturers and students.

Why Agile is Suitable:

- Supports continuous user feedback.

- Accommodates changing requirements.
- Enables incremental development and testing.
- Improves system quality through iterative refinement.
- Reduces development risks by identifying issues early.

KEY MESSAGE
Select a Software Development Methodology because it best supports your project's requirements, not because it is familiar or widely used.

6.4 Research Design

STUDENT QUESTION
During each phase of software development, what information do I need, and how will I obtain it?

STUDENT QUESTION
Which research activities do I need, and at which stage of my project will each one happen?

What is Research Design?

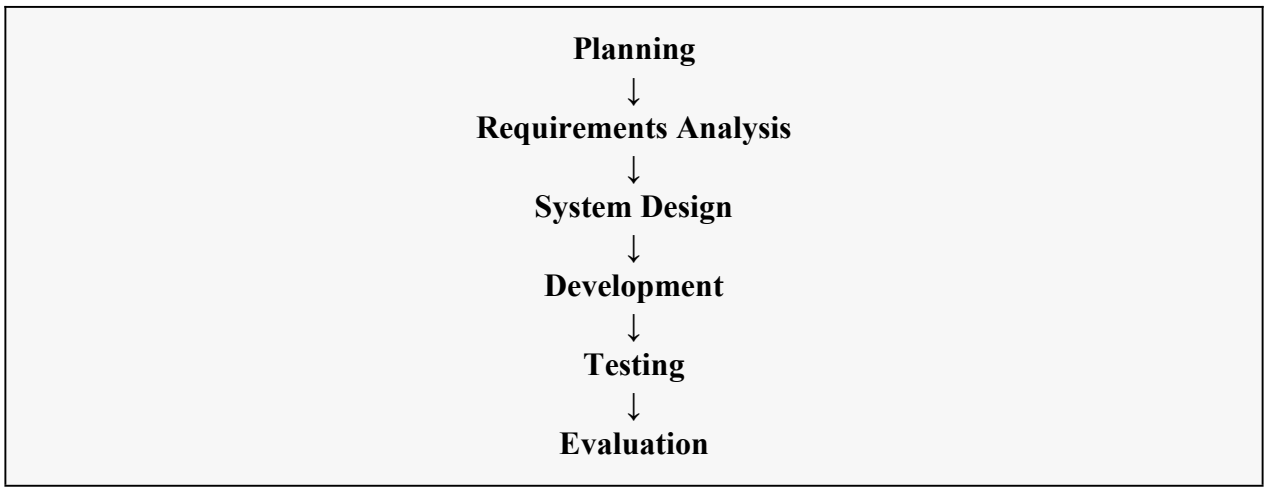
Once the Software Development Methodology has been selected, the next step is to plan how research activities will support the software development process throughout the project.

Research Design means deciding which research activities will be carried out at each stage of the software development process, and why.

Rather than treating research as a single activity completed before development begins, Research Design treats it as an ongoing set of decisions made throughout the project: what evidence is needed at each stage, which method will collect it, and how it will feed into that stage's outcome.

Aligning Research with Software Development

Unlike traditional research, research in Computing and Technology should support every phase of software development rather than being completed before development begins.



SDLC Phase	Research Activity	Example
Planning	Investigate the problem and define the project scope.	Reviewing Techspire College's current attendance procedures to understand the scale of the problem.
Requirements Analysis	Collect user requirements through questionnaires, interviews, observations, or document analysis.	Distributing a questionnaire to lecturers and students, and conducting interviews with academic administrators about attendance recording challenges.
System Design	Review literature, analyse existing systems, and justify design decisions.	Comparing existing facial recognition attendance systems to justify the chosen recognition algorithm and system architecture.
Development	Implement features based on validated requirements.	Building the facial recognition and attendance recording module using the requirements confirmed in the previous phase.
Testing	Collect user feedback and evaluate system performance.	Observing lecturers using the prototype during a class session and asking for feedback on accuracy and ease of use.
Evaluation	Determine whether the proposed solution successfully addresses the identified problem.	Comparing attendance recording time before and after the system was introduced, using the metric identified in the Planning phase.

For most undergraduate Computing and Technology projects, the research activities described above are best understood as investigating a specific organisation, environment, or problem in depth, since the running example focuses on Techspire College. This approach to Research Design keeps every research activity, from literature review to evidence collection to evaluation, tied to the real context in which the software will be used.

What Should Students Write?

For each relevant phase of the SDLC, students should state:

- Which research activity will be carried out (e.g. questionnaire, interview, observation, document analysis, literature review).
- Why that activity is appropriate at that stage.
- What decision or outcome it will inform.

Together, these statements form the Research Design: a plan showing how research activities are deliberately sequenced across the project rather than treated as a single task completed at the start.

Running Example

Project: Smart Facial Recognition-Based Attendance Management System.

Research Design: Research activities are planned throughout the Agile development process, focused on understanding and validating the attendance management problem at Techspire College, to ensure that user feedback continuously improves the system.

SDLC Phase	Research Activity	Purpose
Planning	Document analysis of current attendance procedures.	Understand the scope of the problem at Techspire College.
Requirements Analysis	Questionnaire and interviews with lecturers and students.	Identify specific requirements and concerns.
System Design	Literature review of facial recognition and biometric systems.	Justify the chosen system design.
Development	Regular check-ins and walkthroughs with lecturers during implementation.	Confirm that the system being built still matches the requirements identified earlier.
Testing	Questionnaire and observation during user testing.	Evaluate whether the prototype meets user needs.
Evaluation	Comparing attendance recording time before and after the system was introduced.	Determine whether the proposed solution successfully addresses the problem identified in Planning.

KEY MESSAGE

Research Design explains how research is planned and organised so that it continuously supports software development from planning through evaluation.

6.5 Research Approach

What is a Research Approach?

The Research Approach explains the type of evidence required to answer the research questions and achieve the project objectives. Different projects require different types of evidence. Some require detailed user opinions, while others require measurable numerical data. Many Computing and Technology projects require both.

Purpose of the Research

Closely related to the Research Approach is the purpose the research serves at a given point in the project, since this shapes which methods are most appropriate:

- **Exploratory research** is conducted during the early stages of a project to understand a topic generally, and is useful when the problem is not yet well understood. Example: interviewing a few lecturers informally to understand general frustrations with the current attendance process.
- **Descriptive research** describes or defines a particular phenomenon as it currently exists. Example: using a questionnaire to measure how satisfied users are with the current eDigital Attendance System.
- **Explanatory research** aims to explain why a particular phenomenon occurs the way it does. Example: investigating why response rates to the current manual attendance process are low.

Most undergraduate Computing and Technology projects move through all three purposes as the project progresses: starting exploratory to understand the problem, then descriptive to measure the current situation, and finally explanatory to justify why the proposed solution is needed.

Common Research Approaches

Research Approach	Suitable When	Common Methods	Example
Qualitative	Understanding opinions, experiences, and behaviours.	Interviews, observations, focus groups.	Interviewing lecturers about attendance challenges.
Quantitative	Measuring opinions, behaviours, or trends using numerical data.	Questionnaires and surveys.	Measuring student satisfaction with the current attendance system.
Mixed Methods	Both numerical data and detailed user insights are required.	Combination of qualitative and quantitative methods.	Smart Attendance Management System

Choosing an Appropriate Research Approach

Students should select the approach that best supports their research objectives. For most undergraduate software development projects: Qualitative is useful for exploring user experiences and identifying requirements; Quantitative is useful for measuring user opinions

and validating findings; and Mixed Methods combines the strengths of both approaches and is therefore commonly adopted in Computing and Technology research.

Rather than selecting an approach because it is popular, students should explain why the chosen approach provides the evidence required for their project.

What Should Students Write?

Students should identify the selected Research Approach, explain why it is suitable for the project, describe the type of evidence that will be collected, and justify how it supports the project objectives.

Running Example

The Smart Facial Recognition-Based Attendance Management System adopts a Mixed Methods approach. Quantitative data are collected through questionnaires to measure user satisfaction and identify common attendance challenges. Qualitative data are collected through interviews and observations to understand lecturers' experiences, workflow issues, and expectations for the proposed system.

Combining these methods provides a more comprehensive understanding of user requirements and supports better software design decisions.

BEST PRACTICE
Select the Research Approach based on the evidence your project requires, not simply because it is commonly used.

STUDENT QUESTION
Who can provide the information needed to successfully develop this software?

6.6 Target Users

Who are the Target Users?

Target users are the individuals who will directly use, interact with, or benefit from the proposed software system. They are the primary source of information for understanding user needs, validating requirements, and evaluating the completed system.

Students should ensure that the selected target users are directly relevant to the research problem and the proposed software solution.

Target Users vs Stakeholders

Students often confuse target users with stakeholders.

Category	Description	Running Example
Target Users	Individuals who directly use or interact with the proposed system.	Lecturers and students
Stakeholders	Individuals or organisations who influence or are affected by the system but **may not** use it directly. *All target users are Stakeholders but All Stakeholders might not be target users.*	Academic Administration Staff, Programme Coordinators, College Management

Although stakeholders may not always use the software themselves, they often provide valuable information regarding organisational requirements, policies, and operational processes.

Selecting Appropriate Target Users

When identifying target users, students should consider:

- Who currently experiences the problem?
- Who will use the proposed system?
- Who can provide meaningful information about the problem?
- Who will evaluate the completed system?

Selecting appropriate target users ensures that the collected evidence is relevant and supports the project objectives.

What Should Students Write?

Students should identify each target user group, explain why each group is relevant, describe how each group contributes to the research, and justify why these participants are suitable for the project.

Running Example

User Group	Contribution
Lecturers	Explain current attendance challenges and evaluate usability.
Students	Provide feedback on user experience and system acceptance.
Academic Administration Staff	Explain reporting requirements and institutional procedures.
Programme Coordinators	Provide operational and academic management perspectives.

KEY MESSAGE

Select target users because they can provide meaningful evidence that supports the development of the proposed software system.

STUDENT QUESTION
Which people should I involve, and why are they the most appropriate participants?

6.7 Sampling Strategy

What is Sampling?

Sampling is the process of selecting a subset of participants from the target user population to take part in the research. Since it is often impractical to collect data from every potential user, researchers select participants who can provide reliable and relevant information for the study.

An appropriate sampling strategy helps ensure that the collected evidence accurately represents the needs and experiences of the intended users.

Sampling techniques fall into two broad families.

Probability sampling gives every member of the population a known, non-zero chance of selection, which supports statistical generalisation; Random, Systematic, Stratified, and Cluster Sampling belong to this family.

Non-probability sampling selects participants using judgement or convenience (subjectively) rather than random chance, which is often more practical for smaller academic projects; Convenience, Purposive, Quota, Snowball, and Self-selection Sampling belong to this family.

Common Sampling Techniques

Sampling Technique	Description	Suitable When	Example
Convenience Sampling	Participants are selected because they are easily accessible.	Small academic projects with limited time and resources.	Surveying students available after class.
Purposive Sampling	Participants are selected because they have specific knowledge or experience related to the project.	Research requires participants with relevant expertise or experience.	Selecting lecturers who regularly use the eDigital Attendance System.
Random Sampling	Every member of the target population has an equal chance of being selected.	Large populations where unbiased representation is important.	Randomly selecting students from the entire IT programme.
Stratified Sampling	Participants are selected from different subgroups to ensure balanced representation.	The population contains distinct groups that should all be represented.	Selecting students from each academic year.
Systematic Sampling	Every kth element of the population is selected after a random starting point, using a fixed sampling interval.	A sampling frame exists and a simple, evenly spaced selection is acceptable.	Selecting every 5th student from an enrolment list.
Cluster Sampling	The population is divided into naturally occurring groups (clusters), and entire clusters are randomly selected for study.	The population is spread across distinct groups and surveying every individual is impractical.	Randomly selecting two lecture sections to survey out of all sections.

Sampling Technique	Description	Suitable When	Example
Quota Sampling	Participants are selected to fill fixed quotas representing key subgroups of the population, without random selection.	A sampling frame is unavailable but proportional representation across subgroups is still needed.	Selecting a fixed number of students from each year of study.
Snowball Sampling	Initial participants refer other participants with relevant experience or characteristics, building the sample through referrals.	The target population is hard to identify or reach directly.	Locating students experienced with facial recognition systems through referrals from initial contacts.
Self-selection Sampling	Participants volunteer themselves after the researcher publicises a need for cases, rather than being selected by the researcher.	Willing, motivated respondents are needed and a public call for participants is practical.	Inviting students and lecturers to volunteer for the study via an open call on the college notice board or email list.

For most undergraduate Computing and Technology projects, Purposive Sampling is often the most appropriate because participants possess relevant knowledge or experience. Convenience Sampling may also be suitable when participants are readily available and project resources are limited. Random and Stratified Sampling are generally more appropriate for larger-scale research where broader representation is required.

The chosen sampling strategy should always be justified according to the research objectives rather than selected simply because it is easy to implement.

What Should Students Write?

Students should identify the selected sampling technique, estimate the number of participants, explain why those participants are suitable, and justify why the chosen sampling strategy supports the research objectives.

Running Example

For the Smart Facial Recognition-Based Attendance Management System:

Target Users: Lecturers, students, academic administration staff, and programme coordinators.

Sampling Strategy: Purposive Sampling. Justification: Participants are selected because they regularly interact with the existing eDigital Attendance System and can provide valuable insights into attendance processes, operational challenges, and user requirements.

BEST PRACTICE

Select participants based on their ability to provide meaningful evidence, not simply because they are easy to access.

STUDENT QUESTION
How will I gather the information needed throughout software development?

6.8 Data Collection Methods

What are Data Collection Methods?

After identifying the target users and selecting an appropriate sampling strategy, the next step is to determine how research evidence will be collected. Data Collection Methods are the techniques used to gather information that supports the research objectives and software development process.

Students should select methods based on the type of evidence required rather than using methods simply because they are familiar.

Common Data Collection Methods

Method	Purpose	Strengths	Limitations
Questionnaire	Collect information from a large number of participants efficiently.	Quick to administer, easy to analyse, suitable for quantitative data.	Limited opportunity for detailed responses.
Interview	Explore participant opinions, experiences, and expectations in depth.	Rich qualitative insights and allows follow-up questions.	Time-consuming and involves fewer participants.
Observation	Understand actual user behaviour, workflows, and system usage.	Reveals real practices that users may not describe accurately.	Can be time-consuming and influenced by observer bias.
Focus Group	Generate discussion among multiple participants to explore ideas and requirements.	Provides multiple perspectives and encourages interaction.	Discussions may be dominated by a few participants.
Document Analysis	Review existing documents, reports, policies, or system documentation.	Uses existing organisational evidence and procedures.	Information may be incomplete or outdated.

Using Multiple Methods (Triangulation)

Collecting evidence from different sources helps validate findings and reduces the limitations of individual methods. This process is commonly known as triangulation.

Research Objective	Suitable Method
Understand user experiences	Interview
Measure user satisfaction	Questionnaire
Analyse existing workflows	Observation
Review organisational procedures	Document Analysis

Using multiple methods enables researchers to compare findings, identify consistent patterns, and develop a more comprehensive understanding of the problem.

What Should Students Write?

Students should identify the selected data collection methods, explain why each method is appropriate, describe the type of information each method will collect, and justify how the collected evidence supports the research objectives and software development.

Running Example

Method	Purpose
Questionnaire	Measure lecturer and student satisfaction with the existing attendance process.
Interview	Understand detailed challenges, expectations, and operational requirements.
Observation	Examine how attendance is currently recorded during lectures.
Document Analysis	Review attendance policies and existing eDigital Attendance System procedures.

KEY MESSAGE

Select data collection methods that produce the evidence needed to design and evaluate a better software solution.

STUDENT QUESTION

How can I conduct this research responsibly while developing the software?

6.9 Ethical Considerations

Why are Ethical Considerations Important?

Research involving human participants should always be conducted responsibly and ethically. Ethical considerations protect participants, ensure responsible use of information, and strengthen the credibility of the research findings.

Students should demonstrate that they will collect, store, and use research data in a respectful, secure, and transparent manner.

Common Ethical Principles

Ethical Principle	Explanation
Informed Consent	Participants should voluntarily agree to participate after understanding the purpose of the research.

Ethical Principle	Explanation
Confidentiality	Personal information should not be disclosed without permission.
Privacy	Data should only be collected and used for legitimate research purposes.
Voluntary Participation	Participants should be free to withdraw from the research at any time without penalty.
Data Security	Research data should be stored securely and accessed only by authorised individuals.

Ethical Considerations for Computing Projects

Many Computing and Technology projects involve collecting user information, system logs, photographs, videos, or biometric data. Students should consider how these data will be protected throughout the project. Where personal or sensitive information is collected, students should explain:

- How consent will be obtained.
- How data will be securely stored.
- Who will have access to the data.
- How privacy will be protected.
- How data will be disposed of after the project, where appropriate.

Running Example

The proposed Smart Facial Recognition-Based Attendance Management System uses facial recognition technology. Facial images are considered biometric data, which require additional protection. Students should explain that participants will provide informed consent before facial images are collected; facial images will only be used for research and system testing; biometric data will be securely stored with restricted access; and participant privacy and confidentiality will be maintained throughout the project.

BEST PRACTICE

Ethical considerations should demonstrate not only compliance with research principles but also responsible software engineering practices.

6.10 Running Example : Bringing the Methodology Together

The following example demonstrates how all components of the Research Methodology work together to support evidence-based software development.

Methodology Component	Running Example
Software Development Methodology	Agile
Research Design	Research activities aligned with Agile development phases.
Research Approach	Mixed Methods.
Target Users	Lecturers, Students, Academic Administration Staff, Programme Coordinators.
Sampling Strategy	Purposive Sampling.
Data Collection Methods	Questionnaire, Interview, Observation, Document Analysis.
Research Instruments	Google Forms questionnaire, Interview Guide, Observation Checklist.
Ethical Considerations	Informed consent, confidentiality, privacy, secure handling of biometric data.

This example demonstrates how each component contributes to a structured research process that supports software development from planning to evaluation.

Methodology at a Glance



Testing & Evaluation
↓
Improved Software Solution

This framework reinforces the central philosophy of RMCT: research is a continuous process that supports every stage of software engineering.

Common Mistakes

Students should avoid the following common mistakes when writing the Research Methodology:

- Confusing Software Development Methodology with Research Methodology.
- Selecting research methods without explaining why they are appropriate.
- Choosing a Research Approach because it is popular rather than because it supports the project objectives.
- Identifying target users without justifying the sampling strategy.
- Collecting data from participants who are not relevant to the proposed system.
- Ignoring ethical responsibilities when collecting personal or sensitive data.
- Describing methodology components independently without explaining how they work together.

Best Practice

A strong Research Methodology should:

- align research activities with the Software Development Life Cycle;
- justify every methodological decision using the project objectives;
- collect evidence from appropriate users using suitable methods;
- apply ethical principles throughout the research process; and
- demonstrate how research findings will support software requirements, technical decisions, system development, and evaluation.

Remember that the Research Methodology is not simply a description of research methods. It is the complete roadmap that explains how research and software development work together to produce a successful software solution.

Chapter Summary

After completing this chapter, students should understand that a Research Methodology provides a structured plan for conducting both research and software development; the Software Development Methodology determines how the software will be developed, while the Research Design explains how research supports that development; the Research Approach, Target Users, Sampling Strategy, and Data Collection Methods should all be selected and justified according to the project objectives; ethical considerations protect participants and strengthen the credibility of the research; and all methodology components work together to ensure that software is developed using reliable evidence rather than assumptions.

Ultimately, a well-planned methodology enables researchers to collect meaningful evidence, make informed technical decisions, develop software that meets user needs, and evaluate whether the proposed solution successfully addresses the identified problem.

Transition to Chapter 7

The Research Methodology explains what evidence needs to be collected and how it will be collected. The next step is to focus on how to collect high-quality evidence effectively. In Chapter 7: Collecting Evidence from Users, students will learn how to design effective questionnaires, conduct interviews, perform observations, analyse documents, reduce research bias, and ensure that the evidence collected is reliable, relevant, and useful for software development.

CHAPTER 7

Collecting Evidence from Users

Chapter Purpose

Effective software systems are built on reliable evidence rather than assumptions. This chapter explains how to collect meaningful information from users and stakeholders using appropriate research methods. Students will learn how to design questionnaires, conduct interviews, perform observations, analyse documents, and combine multiple sources of evidence to understand user requirements and support evidence-based software development.

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain the importance of collecting reliable evidence for software development.
- Select appropriate data collection methods for different research objectives.
- Design effective research instruments.
- Recognise the strengths and limitations of different data collection methods.
- Reduce bias and improve the quality of collected evidence.
- Understand how collected evidence contributes to software requirements and system design.

7.1 Why Evidence Collection Matters

Every successful software system begins with understanding the needs of its users. Without reliable evidence, software teams risk several avoidable failures:

- Misunderstanding user requirements.
- Solving the wrong problem.
- Developing unnecessary features.
- Overlooking usability issues.
- Making technical decisions based on assumptions rather than facts.

Evidence collection allows software engineers to understand real-world problems, validate assumptions, and ensure that the proposed solution addresses genuine user needs.

KEY MESSAGE
Good software is built on good evidence.

Literature Review Produces Assumptions, Not a Finished System

It is tempting to think that once the Literature Review has justified a technical direction, the system itself is effectively decided. In reality, the Literature Review produces evidence-based assumptions; informed starting points, not final requirements. For example:

- Facial Recognition is an appropriate technology.
- Role-based access is useful.
- Manual override is valuable.
- Attendance analytics improve monitoring.
- Audit logs improve accountability.

These are reasonable, evidence-backed assumptions. They are not yet confirmed as what this particular organisation actually needs.

Requirements Gathering Does Not Start From Zero

Because the Literature Review has already produced these assumptions, interviews and questionnaires do not begin with a blank page. Instead, Requirements Gathering confirms, refines, extends, prioritises, or occasionally rejects each assumption:

- **Confirm :** Manual override is required.
- **Refine :** Only lecturers may override attendance within 24 hours.
- **Extend :** Integration with Moodle is also needed.
- **Prioritise :** Attendance recording matters more than analytics.
- **Reject :** Push notifications are unnecessary.

Literature Review Improves Research Methodology

One of the most important ideas in this guide: the better a researcher understands the literature, the better questions they ask. Without literature, questions stay vague, such as "What features do you need?" With literature behind them, questions become intelligent and specific.

- Instead of "What reports do you need?" ask "Research shows attendance systems commonly provide daily, weekly, and semester reports. Which of these would support your work?"
- Instead of "What security do you need?" ask "Who should be allowed to modify attendance records? Should modifications be recorded in an audit log?"

KEY MESSAGE
Literature does not replace data collection. Literature improves data collection.

A useful comparison: doctors still interview their patients even though they hold extensive medical knowledge. That knowledge does not make the interview unnecessary, it helps the

doctor ask sharper, more relevant questions. Software engineering research works the same way.

The Real Difference Between Literature Review and Requirements Gathering

Rather than simply "general vs. specific," it is more accurate to say that Literature Review investigates the solution space, while Requirements Gathering investigates the organisation or the problem space.

- Literature Review: What technologies exist? What methodologies exist? What architectures exist? What best practices exist? What research gaps exist?
- Requirements Gathering: How do lecturers currently take attendance? What problems do they experience? What reports are required? Who edits attendance? How should the workflow operate? What constraints exist?

KEY MESSAGE
Literature tells us what CAN be built. Requirements Gathering tells us what SHOULD be built for THIS organisation.

Requirements Are Not Features

Students often equate "requirements" with "features." A requirement is everything a developer needs to know before building the software, not just what it should do, but also its behaviour, workflow, business rules, user expectations, performance, reliability, constraints, and usability.

Six Viewpoints, Then FURPS+

Rather than starting from an acronym, it helps to think of requirements as interviewing the software from six perspectives:

- Functional: What should it DO?
- Usability: How should using it FEEL?
- Reliability: What should happen when things go wrong?
- Performance: How FAST should it work?
- Supportability: How EASY should it be to maintain?
- Plus: What RULES and CONSTRAINTS must developers respect?

Once these six perspectives are understood, they map directly onto the standard acronym: FURPS+.

The "+" deserves its own attention, since it covers several distinct kinds of constraint: Design Constraints, Interface Constraints, Hardware Constraints, Software Constraints, Business Constraints, Legal Constraints, and Ethical Constraints. For example: must integrate with eDigital; must use existing classroom cameras; must comply with the institution’s privacy policy; must run on existing infrastructure.

What Questions Should Students Ask?

Questions should uncover workflow, business rules, exceptions, constraints, and usability, performance, and reliability expectations, not technology preferences.

Weaker questions: "Should we use AI?" / "Should we use Agile?" / "Should we use MySQL?" / "Should we use Facial Recognition?"

Stronger questions: "How is attendance currently taken?" / "What problems waste the most time?" / "When should attendance be recorded?" / "Who may edit attendance?" / "What reports are required?" / "What happens if recognition fails?" / "Should attendance be completed within one minute?"

Important Exception

Technology decisions normally come from the literature. However, Requirements Gathering sometimes surfaces privacy concerns, budget limits, existing infrastructure, institutional policy, or legal issues that the literature did not anticipate. These findings do not invalidate the Literature Review, they simply trigger another round of research, exactly as described earlier in the "re-search" idea.

7.2 Selecting Appropriate Data Collection Methods

Different research questions require different methods of collecting evidence. There is no single method that is suitable for every project. The selection should depend on:

- Project objectives.
- Research questions.
- Target users.
- Available resources.
- Project scope.
- The type of information required.

Method	Best Used When	Type of Evidence
Questionnaire	Information is required from many participants efficiently.	Quantitative and limited qualitative data
Interview	Detailed opinions and experiences are required.	Primarily qualitative (quantitative if structured with closed items)
Observation	Actual user behaviour and workflows need to be understood.	Qualitative and Quantitative
Focus Group	Multiple participants discuss ideas and requirements together.	Qualitative
Document Analysis	Existing policies, procedures, or records provide useful information.	Qualitative and Quantitative (depends on document type)

BEST PRACTICE
Students should choose methods that best support their research objectives rather than selecting methods simply because they are familiar.

7.3 Designing Effective Questionnaires

Questionnaires are one of the most commonly used methods in undergraduate software development projects because they enable researchers to collect information from a relatively large number of participants efficiently.

A well-designed questionnaire should follow several practical guidelines:

- Use simple and clear language.
- Avoid leading or biased questions.
- Focus directly on the research objectives.
- Include an appropriate balance of closed and open-ended questions.
- Follow a logical sequence.

Example topics for the running project include:

- Current attendance recording experience.
- Time required to mark attendance.
- Common challenges encountered.
- Satisfaction with the existing eDigital Attendance System.
- Opinions on AI-based facial recognition.
- Concerns about privacy and security.

BEST PRACTICE
Every questionnaire question should help answer a research objective or support a future software requirement.

Likert Scale Questions

Many software projects use Likert scales to measure user perceptions and attitudes. For example, the item "The current attendance recording process is efficient" could use response options such as Strongly Disagree, Disagree, Neutral, Agree, and Strongly Agree. Likert scales make responses easier to analyse statistically.

BEST PRACTICE
Use consistent response scales throughout the questionnaire.

Types of Questionnaire Items

Beyond Likert-style rating questions, questionnaires typically draw on a small set of question types, each suited to a different kind of information:

- **List questions** offer a set list of responses the participant chooses from (e.g. Yes/No, Agree/Disagree). Example: "Have you previously experienced problems using the eDigital Attendance System? Yes / No."
- **Category questions/Multiple Choice questions** ask the participant to select the one option that best fits, useful for classifying respondents (e.g. role: lecturer, student, administrator). Example: "Which best describes your role at Techspire College? Lecturer / Student / Administrative Staff / Programme Coordinator."
- **Dichotomous questions** restrict the respondent to exactly two choices, typically Yes/No, and are the simplest form of List question. They are normally used to obtain a quick, unambiguous response without further explanation. Example: “Have you previously experienced problems using the eDigital Attendance System? Yes / No.”
- **Ranking questions** ask participants to place items in order of importance, useful for understanding relative priorities. Example: "Rank the following attendance system features from most to least important: Speed of recording, Accuracy, Privacy protection, Ease of use."
- **Rating questions** (Likert-style) measure strength of agreement or opinion on a numbered scale. Example: "The current attendance recording process is efficient: Strongly Disagree / Disagree / Neutral / Agree / Strongly Agree." Example 2: “Rate the current attendance system from 1 (Very Inefficient) to 5 (Very Efficient).”
- **Quantity questions** ask for a numeric amount (e.g. how many times per week). Example: "On average, how many minutes does it take to complete attendance recording for one class?"
- **Grid/matrix questions** record responses to several similar questions at once, saving space and respondent effort. Example: rating Speed, Accuracy, and Ease of Use of the current system, each on the same Strongly Disagree to Strongly Agree scale within a single grid.

Selecting the right question type for each piece of information keeps the questionnaire focused and makes the resulting data easier to analyse.

Open-Ended vs. Closed-Ended Questions

Before choosing a specific item type from the list above, it is useful to understand the two broad families that all questionnaire questions fall into: open-ended and closed-ended.

- **Open-ended questions** allow the respondent to answer freely in their own words, without being limited to a fixed set of options. They encourage participants to explain, describe, or reflect, and are useful for uncovering issues the researcher had not anticipated. Example: “How would you describe your experience using the current attendance recording process?”

- **Closed-ended questions** restrict the respondent to a small number of predetermined choices, which makes responses faster to answer and easier to compare and analyse statistically. Closed-ended questions take several forms, most already introduced above under different names:

In practice, most undergraduate questionnaires combine both families: closed-ended questions for quick, easily analysed data, together with a small number of open-ended questions to capture context and issues the researcher had not anticipated.

BEST PRACTICE
Use closed-ended questions for the bulk of the questionnaire so responses are easy to analyse, and reserve open-ended questions for the few issues that need the respondent's own words.

Types of Questionnaire and Distribution Methods

Questionnaires can also be classified by how they are administered, which affects both the effort required and the quality of responses collected.

- **Self-administered questionnaires** are designed to be completed by the respondent without the researcher present, for example an emailed form or an online electronic form. They are convenient for reaching a large number of respondents but offer no opportunity for the researcher to clarify a confusing question.
- **Interviewer-administered questionnaires** are read out and recorded by the researcher, either face-to-face as a structured interview or by telephone. They allow the researcher to clarify questions and reduce misunderstanding, but require more time per respondent and are harder to scale to a large sample.

Self-administered questionnaires can be distributed in several ways, including by post, by hand, by fax, by e-mail, or online using survey tools such as Google Forms, SurveyMonkey, or Microsoft Forms.

BEST PRACTICE
For most undergraduate Computing and Technology projects, an online self-administered questionnaire (e.g. Google Forms) offers the best balance of reach, cost, and ease of analysis.

Key Issues in Survey Research

Before distributing a questionnaire, students should consider several practical issues that affect the quality of the results:

- The effort required to design, produce, and distribute the questionnaire.
- The risk of poor response rates, which are often lower when questionnaires are sent externally than when distributed internally with organisational support.
- The difficulty of writing questions that are clear, unambiguous, free of jargon, and answerable without the respondent needing to seek out extra information.
- The risk of bias if the responding sample is small or dominated by strongly opinionated participants.

These issues reinforce why piloting a questionnaire before full data collection, discussed later in this guide, is considered good practice.

7.4 Conducting Effective Interviews

Interviews provide deeper insights into user experiences, expectations, and challenges. Unlike questionnaires, interviews allow researchers to ask follow-up questions and clarify responses.

For the running example, interviews with lecturers and academic administrators may explore challenges experienced during attendance recording, limitations of the current system, desired system improvements, expectations for automation, and concerns regarding implementation and usability.

Interviews often reveal valuable information that cannot be captured through structured questionnaires alone.

Interview Categories

Students should also state which category of interview they will conduct and why, since this affects how the data can be analysed:

- **Structured interviews** use a fixed, predetermined set of questions asked in the same order and wording to every participant, producing data that is easier to compare across respondents. Example: asking every lecturer the same fixed question, "What challenges do you experience when using the current eDigital Attendance System?"
- **Semi-structured interviews** combine a small number of predetermined questions with follow-up questions that vary between interviews, allowing deeper exploration while still covering the same core themes. Example: asking "What problems have you faced recording attendance?" and then following up with "Can you describe a specific time this caused a delay?" only when relevant.
- **Unstructured interviews** have no predetermined question list and allow the conversation to develop freely, which suits early, exploratory stages of a project but

produces data that is harder to compare across participants. Example: opening with "Tell me about how attendance is currently managed in your department" and letting the conversation guide which topics are explored.

Interviews may also be conducted **one-to-one**, which is typical for in-depth individual accounts, or **one-to-many** as a small group discussion, which can surface a wider range of views in a single session.

For most undergraduate Computing and Technology projects, a semi-structured, one-to-one interview offers a practical balance between comparability and depth.

7.5 Using Observation to Understand Current Processes

Observation enables researchers to understand how work is actually performed rather than relying solely on participant opinions. For the running example, observation may involve attending classes to examine how attendance is currently recorded, how much teaching time is interrupted, how lecturers interact with the eDigital Attendance System, opportunities for automation, and potential operational challenges.

Observation helps researchers identify workflow issues that users may overlook or find difficult to describe.

7.6 Document Analysis

Many organisations already possess valuable information in existing documents, for example, attendance policies, user manuals, workflow diagrams, organisational procedures, system documentation, and institutional reports. Analysing these documents helps researchers understand current processes before proposing system improvements.

For the running example, reviewing Techspire College's attendance procedures and current eDigital Attendance workflow would provide additional evidence to support the proposed solution.

Focus Groups

A focus group brings several participants together to discuss a topic in a facilitated group setting. Focus groups are useful for gathering diverse viewpoints, encouraging interaction among participants, generating new ideas, and identifying shared challenges. For the running example, a focus group with student representatives could explore shared concerns about privacy and the acceptance of facial recognition technology. Focus groups are particularly useful during the early stages of requirements gathering.

7.7 Combining Multiple Methods (Triangulation)

Collecting evidence from a single source may produce incomplete or biased results. A stronger approach is to combine multiple data collection methods so that findings can be compared and validated. This is commonly referred to as triangulation.

Research Objective	Data Collection Method
Understand lecturer challenges	Interview
Measure user satisfaction	Questionnaire
Observe current attendance workflow	Observation
Review institutional attendance procedures	Document Analysis

KEY MESSAGE
Different methods provide different perspectives. Combining them creates a more complete understanding of the problem.

7.8 Designing Research Instruments

A research instrument is the tool used to collect data, for example, questionnaire forms, interview guides, observation checklists, focus group discussion guides, and document review templates. Each instrument should be carefully designed to ensure that the collected evidence is relevant, reliable, and aligned with the research objectives.

Students should avoid creating unnecessary questions that do not contribute to the project.

7.9 Ensuring Data Quality

Reliable evidence depends not only on the selected method but also on the quality of the collected data. Students should consider:

- The clarity of questions.
- Consistency during data collection.
- Participant understanding.
- Completeness of responses.
- Accurate recording of information.

Where possible, questionnaires and interview guides should be reviewed or pilot tested before full data collection begins.

Piloting the Questionnaire

Piloting serves two purposes: to refine the questionnaire so that there are no problems answering or recording the questions, and to assess the validity and reliability of the instrument before it is used for full data collection. A useful way to do this is to ask a few

experts or peers to comment on the representativeness and suitability of the questions, and to allow them to suggest changes to the structure.

As a general guide for undergraduate projects, a pilot of around 10 respondents is normally sufficient for a small student questionnaire, while a large-scale survey may be piloted with 100 to 200 respondents.

When reviewing pilot responses, students should check for the following:

- Were the instructions clear?
- Was the layout clear and attractive?
- Were there any major topic omissions?
- How long did it take respondents to complete the questionnaire?
- Which, if any, questions were unclear or ambiguous?
- Which, if any, questions did the respondent feel uneasy about answering?

BEST PRACTICE

Conduct a small pilot test with a few participants to identify unclear questions and improve your research instruments before collecting the full dataset.

7.10 Recognising Research Bias

Bias occurs when research findings are influenced by factors that reduce objectivity. Common sources of bias include:

- Leading questions, for example, "Don’t you agree that facial recognition would improve attendance recording?"
- Selecting only participants who support the researcher’s expectations.
- Interpreting responses subjectively.
- Ignoring contradictory evidence.

Researchers should remain objective throughout the study and report findings honestly, even if they differ from initial expectations.

COMMON MISTAKE

Designing questionnaires or interviews that encourage participants to provide the answers the researcher expects.

Basic Data Analysis

Collecting data is only the beginning. Students should analyse the evidence to identify patterns and insights that inform software requirements. How this is done depends on whether the data collected is quantitative or qualitative.

Quantitative Data

Quantitative data, typically collected through closed-ended questionnaire items, is best summarised using descriptive statistics and simple visuals:

- **Frequencies:** a count of how many respondents selected each response option, useful for closed-ended questions such as Yes/No or Multiple Choice items. Example: 32 out of 40 lecturers selected "Yes" when asked whether they had experienced attendance recording delays.
- **Percentages:** frequencies converted into a proportion of the total sample, making results easier to interpret and compare across groups of different sizes. Example: 80% of lecturers reported experiencing delays.
- **Mean scores:** the average of a set of numeric responses, commonly used to summarise Likert-scale or rating questions. Example: the average satisfaction rating for the current eDigital Attendance System was 2.3 out of 5.
- **Charts:** visual summaries such as pie charts or bar charts, used to present frequencies or percentages at a glance.
- **Graphs:** visual summaries such as line graphs, useful for showing trends or comparisons across categories, for example attendance recording time before and after introducing the current system.

Qualitative Data

Qualitative data, typically collected through open-ended questionnaire items, interviews, observation notes, and document analysis, is analysed by reading through the responses and identifying patterns rather than by calculation:

- **Common themes:** ideas or issues that are repeated across multiple respondents and grouped together under a descriptive label. Example: several lecturers separately mentioning that manual attendance recording interrupts the start of class.
- **User opinions:** the views, attitudes, and preferences respondents express about the current system or a proposed solution. Example: a lecturer explaining that they would welcome an automated system.
- **Recurring problems:** specific difficulties or limitations mentioned repeatedly across respondents, indicating a genuine pain point rather than an isolated experience.
- **Suggestions:** ideas or recommendations proposed directly by respondents, which can be considered when defining system requirements.

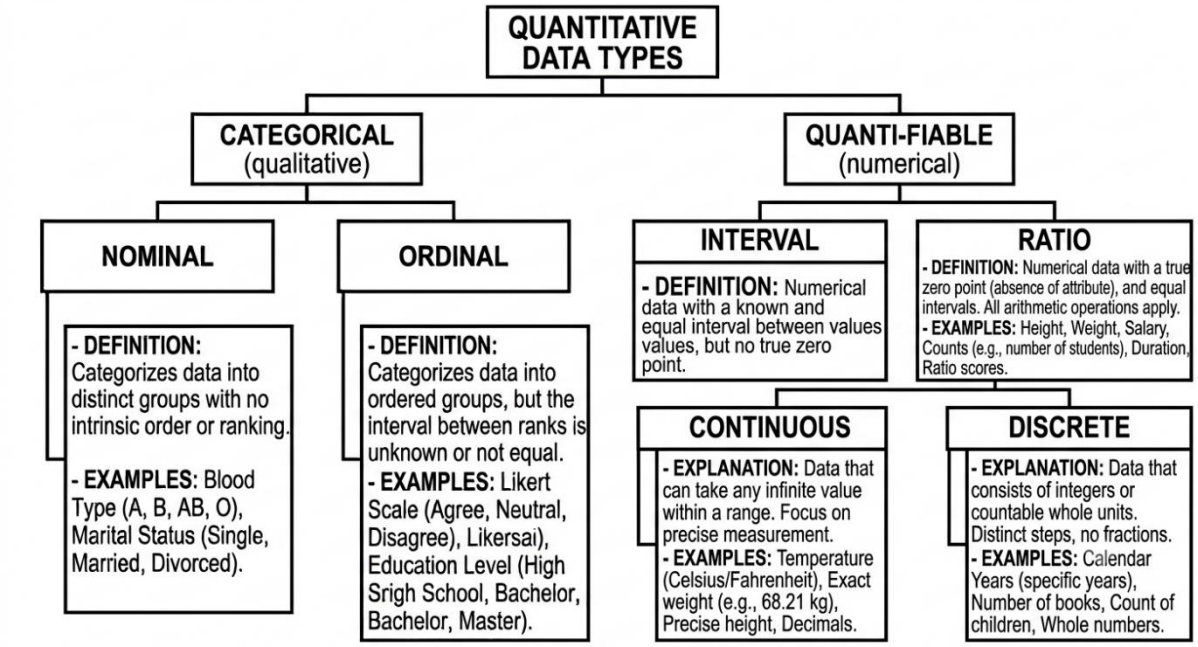
Types of Quantitative Data

The appropriate statistical technique for quantitative data depends on which of the following types has been collected. Data is either **categorical**, which cannot be measured numerically, or **quantifiable**, which can.

Categorical data is further divided into **nominal** data, which labels responses into unordered groups (e.g. gender, department), and **ordinal** data, which places responses in a meaningful order without equal spacing between points (e.g. a Likert scale response of Agree vs Strongly Agree).

Quantifiable data is divided into **interval** data, measured on a scale with equal spacing but no true zero (true zero signifies a total absence of the variable being measured) (e.g. Temperature in Celsius or Fahrenheit, IQ scores, and calendar years), and **ratio** data, which has a true zero point and allows meaningful comparison of magnitude (e.g. number of minutes spent recording attendance, Height, weight, salary, and duration (time)).

Quantifiable data can also be described as **discrete**, meaning it is countable in whole units (e.g. number of students present), or **continuous**, meaning it can take any value within a range (e.g. exact time taken to record attendance).



Choosing an appropriate statistical technique depends on three factors:

- **Research objectives and questions:** the technique chosen must directly answer what the study set out to investigate. For example, if the objective is to compare satisfaction between lecturers and students, the analysis must allow that comparison, rather than simply describing each group in isolation.
- **Measurement scale used in the research instrument** (nominal, ordinal, interval, or ratio): the scale determines which statistical operations are meaningful. Nominal and ordinal data cannot be meaningfully averaged using the arithmetic mean, since the numbers assigned to categories or ranks do not represent equal, measurable distances.

Interval and ratio data, by contrast, can be summarised using the mean, because the distance between values is consistent and meaningful.

- **Overall research design of the study:** for example, whether the study compares two independent groups, tracks the same participants over time, or simply describes a single population, since this affects which technique is appropriate. For instance, comparing satisfaction scores between lecturers and students (two independent groups) calls for a technique that tests the difference between two separate samples, whereas measuring the same lecturers' satisfaction before and after introducing the proposed system would call for a technique suited to paired, repeated measurements on the same participants.

To illustrate, a Likert-scale item asking lecturers to rate their agreement with a statement produces ordinal data: the response options (e.g. Disagree, Neutral, Agree) are ordered, but the distance between them is not guaranteed to be equal. Such data is typically summarised using frequencies, percentages, or the median rather than the arithmetic mean, since the mean assumes an equal spacing between values that ordinal data does not have. By contrast, ratio data such as the number of minutes spent recording attendance can meaningfully be summarised using the mean, because equal numeric intervals genuinely represent equal differences in the underlying value.

7.11 Running Example

The evidence collection process for the Smart Facial Recognition-Based Attendance Management System may be summarised as follows:

Research Objective	Method	Evidence Collected
Identify current attendance challenges	Observation	Existing workflow and operational issues
Measure lecturer and student satisfaction	Questionnaire	Quantitative user feedback
Understand detailed user requirements	Interview	Qualitative insights and expectations
Review institutional procedures	Document Analysis	Existing policies and attendance processes

The collected evidence is then analysed to identify user requirements, validate research findings, justify software features, support system design, and evaluate the final solution.

Common Mistakes

- Selecting data collection methods without considering the research objectives.
- Asking irrelevant or ambiguous questions.
- Collecting data from participants who are not representative of the target users.
- Relying on only one source of evidence when multiple methods are appropriate.
- Ignoring contradictory findings that challenge initial assumptions.

Chapter Summary

After completing this chapter, students should understand that reliable evidence is essential for successful software development; different data collection methods serve different research purposes; research instruments should be carefully designed to collect meaningful information; combining multiple methods strengthens the credibility of research findings; and high-quality evidence supports accurate requirements gathering, informed technical decisions, and effective software design.

Transition to Chapter 8

By this stage, students have investigated the problem, reviewed existing knowledge, identified research gaps, justified technical decisions, planned the research methodology, and collected evidence from users. The final step is to connect all of these activities into a complete software development journey. Chapter 8: Connecting Research to Software Development demonstrates how research findings are transformed into software requirements, system design, implementation, testing, evaluation, and ultimately a successful Final Year Project.

CHAPTER 8

Connecting Research to Software Development

Chapter Purpose

Research and software development are not separate activities. In Computing and Technology, research provides the evidence that guides every stage of software development, from understanding the problem to evaluating the final solution. This chapter brings together everything covered throughout the guide and demonstrates how research continuously supports software development. Students will also understand how the RMCT proposal becomes the foundation for the Final Year Project (FYP).

Learning Outcomes

By the end of this chapter, students should be able to:

- Explain how research supports every stage of software development.
- Connect literature review, research methodology, and software engineering into one continuous process.
- Understand how research findings become software requirements and design decisions.
- Recognise the relationship between RMCT and the Final Year Project.
- Appreciate the importance of evidence-based software engineering.

8.1 The Complete Research Journey

Throughout this guide, students have explored different aspects of research and software development. These activities are not isolated tasks. They form one continuous journey that transforms a real-world problem into a validated software solution.



↓
Research Methodology
↓
Context specific Evidence Collection
↓
Software Requirements
↓
System Design
↓
Software Development
↓
Testing
↓
Evaluation
↓
Improved Software Solution

This journey represents the philosophy of Research for Computing and Technology, where research continuously supports software development rather than occurring only at the beginning of the project.

KEY MESSAGE
Research is not a phase before software development, it is the evidence-based process that supports every stage of software development.

The Final Mindset

Every phase both uses evidence and creates new evidence. Earlier phases mostly rely on existing knowledge, meaning literature and prior systems, while later phases increasingly generate project-specific knowledge, meaning requirements, prototypes, testing, and evaluation. When new evidence challenges earlier assumptions, the researcher revisits the literature, refines the solution, and continues the cycle. Students should finish this guide believing:

- Research is continuous, not a one-time activity.
- Literature Review justifies technical direction; Research Methods/Requirements Gathering customises that direction for the client.
- Existing knowledge and project-specific knowledge are equally important.
- Good literature leads to better data collection questions.
- Good requirements gathering (Research Methodology) confirms, refines, extends, prioritises, or occasionally rejects evidence-based assumptions (Literature Review).
- Every important software development decision should be supported by the best available evidence.

- When new evidence contradicts earlier assumptions, returning to the literature is a strength of research, not a mistake.

8.2 How Research Supports the Software Development Life Cycle

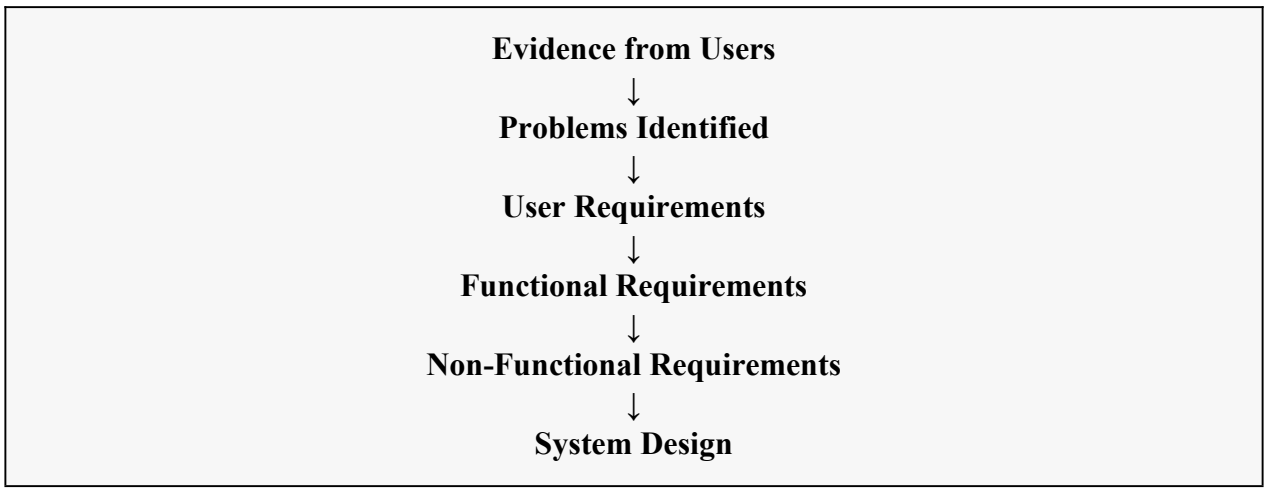
Every phase of software development benefits from research.

SDLC Phase	How Research Contributes
Planning	Investigates the problem, identifies stakeholders, and defines project scope.
Requirements Analysis	Collects user needs through questionnaires, interviews, observations, and document analysis.
System Design	Reviews literature, evaluates technologies, and justifies design decisions.
Development	Implements validated requirements using appropriate technologies.
Testing	Collects user feedback and evaluates system performance.
Evaluation	Determines whether the proposed solution successfully addresses the identified problem and recommends future improvements.

This alignment demonstrates that research remains relevant throughout the entire software development process.

8.3 Transforming Research into Software Requirements

Research findings become the foundation for software requirements.



Research Finding	Software Requirement
Lecturers spend excessive time recording attendance.	The system shall automatically record attendance using facial recognition.
Students expect faster attendance verification.	The system shall verify attendance within a few seconds.
Administrators require attendance reports.	The system shall generate real-time attendance analytics and reports.

This demonstrates that software requirements should emerge from research findings rather than assumptions.

8.4 Running Example: The Complete Journey

The running project demonstrates how each stage contributes to the development of the proposed solution.

Problem

Lecturers manually record attendance using the existing eDigital Attendance System.

Research

Literature review and similar system analysis identify automation technologies and existing limitations.

Research Gap

Current systems do not fully automate attendance recording and provide limited intelligent features.

Technical Decisions

Evidence supports the use of AI-based facial recognition integrated with the existing attendance system.

Methodology

Agile development combined with a Mixed Methods research approach supports iterative software development and continuous user feedback.

Evidence Collection

Questionnaires, interviews, observations, and document analysis identify user requirements.

Development

The system is designed and implemented based on validated requirements.

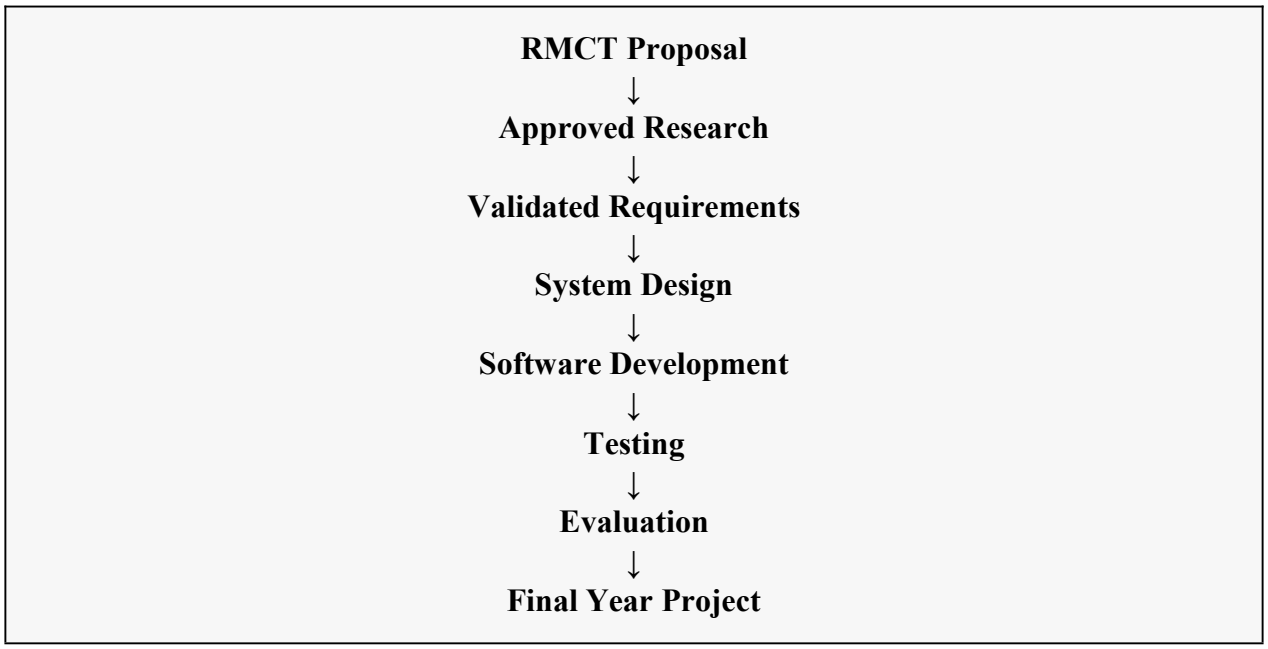
Testing and Evaluation

Users evaluate the completed system to determine whether it improves efficiency, reduces lecturer workload, and addresses the identified problems.

This example demonstrates how research supports every stage of software development.

8.5 RMCT as the Foundation of the Final Year Project

Many students mistakenly believe that RMCT ends when the proposal is submitted. In reality, the proposal becomes the starting point for the Final Year Project.

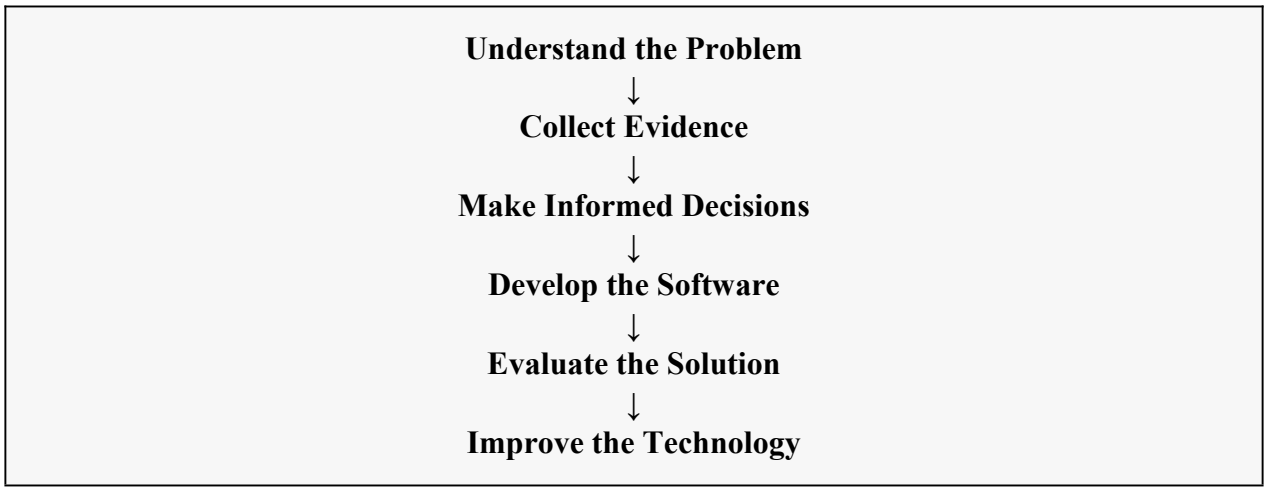


The proposal developed during RMCT provides a clearly defined problem, justified project objectives, evidence from literature, identified research gaps, validated user requirements, appropriate technical decisions, and a structured research methodology. These become the foundation for designing, implementing, testing, and evaluating the Final Year Project.

KEY MESSAGE
A well-prepared RMCT proposal significantly improves the quality and success of the Final Year Project.

8.6 The Philosophy of Research in Computing and Technology

Throughout this guide, one central philosophy has remained consistent. Research in Computing and Technology is not conducted simply to produce knowledge. Instead, research is conducted to understand problems, justify decisions, support software development, and evaluate whether technology effectively addresses real-world needs.



This continuous cycle reflects professional software development practice and encourages lifelong learning and continuous improvement.

Common Mistakes

- Treating research as a task completed before software development begins.
- Developing software without collecting sufficient evidence from users.
- Designing features that are not supported by research findings.
- Ignoring evaluation after system implementation.
- Viewing RMCT and the Final Year Project as unrelated modules.

Best Practice

Throughout your project, continually ask yourself:

- What problem am I trying to solve?
- What evidence supports this decision?
- What do users actually need?
- How does this research improve my software?
- Does my final system solve the identified problem?

Keeping these questions in mind will help ensure that your project remains focused, evidence-based, and aligned with the objectives of Computing and Technology research.

Final Chapter Summary

By completing this guide, students should understand that research and software development are closely integrated; literature review, similar system analysis, and research methodology collectively support software engineering; research findings become user requirements, technical decisions, and system designs; software should be developed based on evidence rather than assumptions; testing and evaluation determine whether the proposed solution effectively addresses the identified problem; and the RMCT proposal serves as the research and planning foundation for the Final Year Project.

Final Reflection

The central philosophy of this guide can be summarised in one statement:

FINAL RMCT PHILOSOPHY

Research in Computing and Technology is a continuous, evidence-based process of investigating problems, understanding users, analysing existing knowledge, making informed technical decisions, developing software solutions, and evaluating whether those solutions effectively solve real-world problems.

Throughout this guide, one distinction is worth carrying forward above all others:

KEY MESSAGE

Research is not a chapter of the proposal. Research is the continuous process that guides every important software development decision using evidence.
Research ≠ Literature Review. Research = evidence-based decision making throughout the SDLC.

Students should carry this philosophy throughout their RMCT module and into their Final Year Project.

A successful proposal is not merely a document submitted for assessment, it is the blueprint for building a well-researched, well-designed, and well-evaluated software solution.

Bibliography

Asia Pacific University of Technology & Innovation. (2026). *Research Methods for Computing and Technology (RMCT) assignment description* [Assignment description]. APU Moodle. apu.edu.my

Asia Pacific University of Technology & Innovation. (2026). *Research Methods for Computing and Technology (RMCT) lecture slides* [PowerPoint slides]. APU Moodle. apu.edu.my

Asia Pacific University of Technology & Innovation. (2026). *Research Methods for Computing and Technology (RMCT) module descriptor* [Module descriptor]. APU Moodle. apu.edu.my

OpenAI. (2026). *ChatGPT* (GPT-5.5, July 2026 version) [Large language model]. https://chatgpt.com

(Used for brainstorming, draft content generation and organization, and polished presentation development. All content was reviewed, verified, and adapted by the Lecturer.)